

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
ИНСТИТУТ МАТЕМАТИКИ И ИНФОРМАЦИОННЫХ СИСТЕМ
ФАКУЛЬТЕТ КОМПЬЮТЕРНЫХ И ФИЗИКО-МАТЕМАТИЧЕСКИХ НАУК
КАФЕДРА ПРИКЛАДНОЙ МАТЕМАТИКИ И ИНФОРМАТИКИ

Допущен к защите

Заведующий кафедрой ПМИ

_____ Е.В. Разова

РАЗРАБОТКА БД И ПРИЛОЖЕНИЯ ДЛЯ УЧЕТА ПРОДАЖИ ПРЕДМЕТОВ ИСКУССТВА

Курсовой проект по дисциплине
«Базы данных»

Выполнил студент группы ПМИб-3301-52-00 _____ / П.В. Рылова /

Руководитель ст. преподаватель кафедры ПМИ _____ / И.Н. Фищева /

Работа защищена с оценкой _____ _____.2025 г.

Члены комиссии: _____ / _____ /
_____ / _____ /

Содержание

Введение.....	3
1 Обзор аналогов и разработка требований к информационной системе.....	5
1.1 Обзор аналогов	5
1.2 Разработка требований к информационной системе.....	8
1.2.1 Бизнес-требования.....	8
1.2.2 Бизнес-правила	8
1.2.3 Пользовательские требования.....	9
1.2.4 Ограничения.....	10
2 Проектирование информационной системы.....	11
2.1 Выбор и обоснование архитектуры ИС и используемого ПО ...	11
2.2 Объектно-ориентированное проектирование	15
2.3 Проектирование базы данных.....	21
3 Разработка информационной системы	26
3.1 Разработка бизнес-логики на уровне СУБД.....	26
3.2 Разработка бизнес-логики на уровне приложения	28
3.3 Разработка интерфейса ИС	30
Заключение	37
Библиографический список	38
Приложения	40
Приложение А. Листинг программы	40
Приложение Б. Перечень сокращений и обозначений	46

Введение

В условиях цифрового развития особую актуальность приобретает создание удобных онлайн-платформ, автоматизирующих процессы купли-продажи. Такие системы позволяют продавцам легко размещать товары или находить клиентов, а покупателям – быстро находить, заказывать и оплачивать понравившиеся товары. Хотя сегодня существуют много маркетплейсов, они недостаточно гибкие для индивидуальных заказов и не предоставляют продавцам возможностей для детального учёта продаж. Поэтому нужны специализированные платформы, например, для рынка искусства.

Таким образом, проблема исследования заключается в необходимости разработки информационной системы, обеспечивающей автоматизированный учет продаж предметов искусства.

Целью курсового проекта является разработка базы данных и приложения для учёта продажи предметов искусства.

Для достижения цели необходимо решить следующие **задачи**:

- провести анализ существующих аналогов информационных систем;
- разработка функциональных и нефункциональных требований к системе;
- спроектировать архитектуру приложения и базы данных;
- разработать программную реализацию бизнес-логики на уровне базы данных и приложения;
- разработать пользовательский интерфейс.

Курсовой состоит из четырех разделов. В первом разделе раскрываются актуальность темы, цель и задачи работы и обзор аналогов существующих решений в данной предметной области.

Во втором разделе формулируются требования к информационной системе, включая функциональные и нефункциональные аспекты.

Третий раздел посвящён проектированию системы, выбору архитектуры, объектно-ориентированному моделированию и проектированию базы данных.

В четвертом разделе описывается процесс разработки информационной системы, включая реализацию бизнес-логики и интерфейса.

1 Обзор аналогов и разработка требований к информационной системе

1.1 Обзор аналогов

На современном рынке существует ряд информационных систем, предназначенных для продажи предметов искусства и авторских изделий. Ниже приведён анализ наиболее известных платформ:

Etsy – международная платформа для реализации изделий ручной работы и художественных работ.

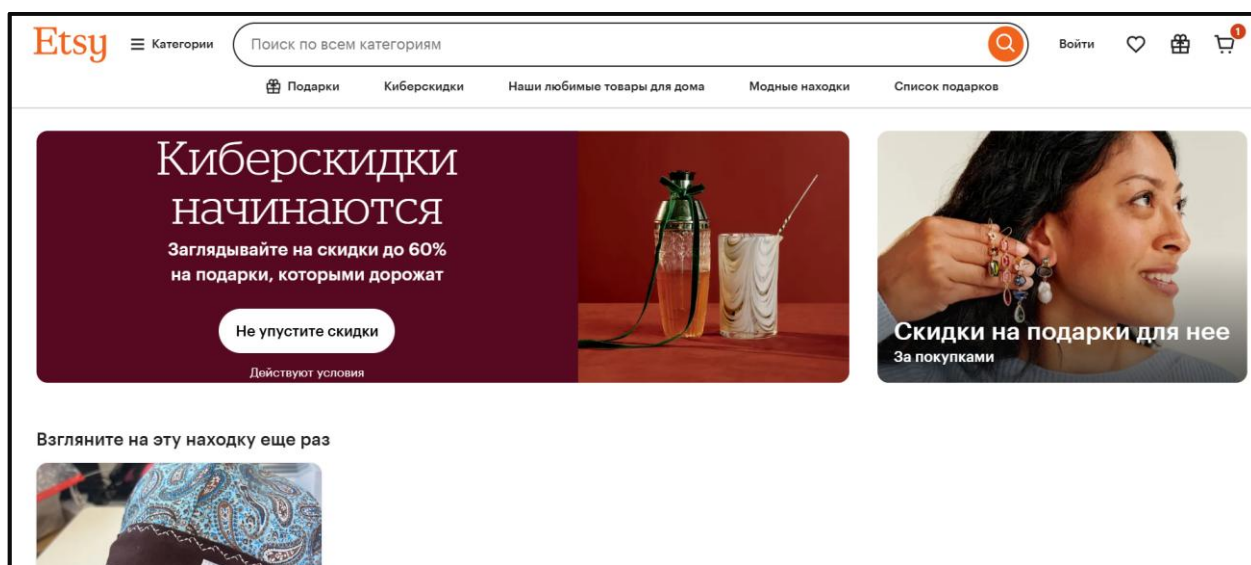


Рисунок 1.1 – Интерфейс Esty

Достоинства:

- наличие структурированной модели данных для товаров (каталоги, цены, изображения, теги, названия);
- поддержка стандартных операций учёта: заказы, статусы, платёжные транзакции;
- данные о товарах и заказах доступны через API, но без подробной информации.

Недостатки:

- отсутствие полного доступа к базе данных;
- все транзакции привязаны к внутренней логике платформы, без возможности расширения;

- невозможность локального хранения данных;
- отсутствие аналитических инструментов: нет возможности строить отчеты по динамике продаж, остаткам, категориям или авторам.

Saatchi Art – специализированная онлайн-галерея, ориентированная на профессиональных художников.

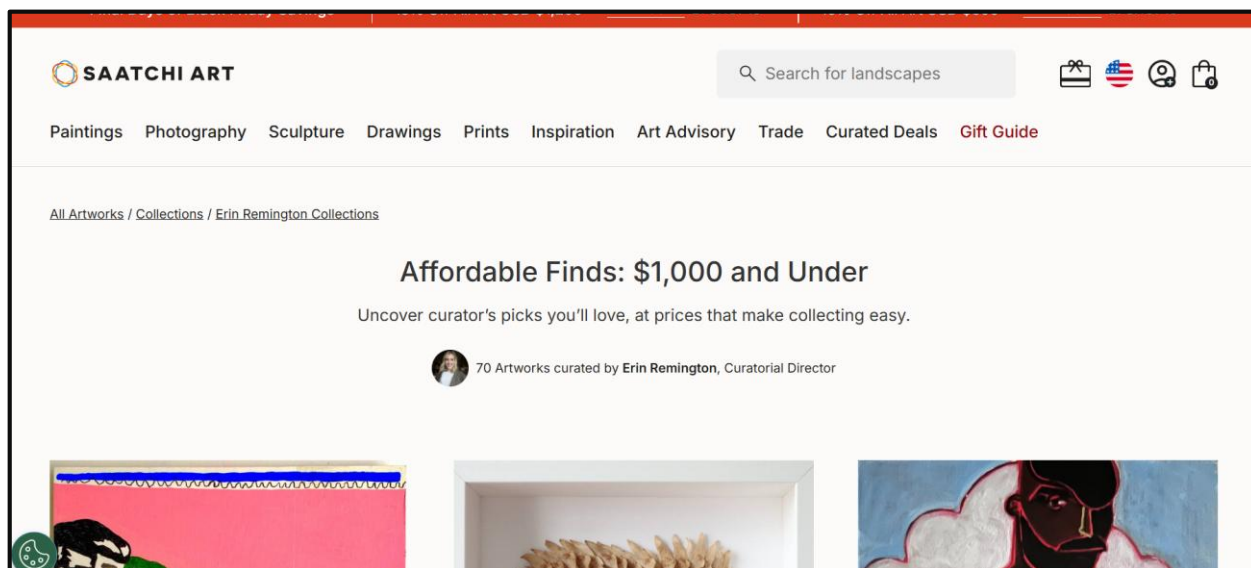


Рисунок 1.2 – Интерфейс Saatchi Art

Достоинства:

- расширенная структура описания товара;
- поддержка сложных бизнес-процессов;
- обеспечивает полный цикл продажи.

Недостатки:

- полное отсутствие API и экспорта данных (информация о клиентах, сделках, товарах недоступна для внешнего использования);
- архитектура платформы не предусматривает синхронизацию с локальными системами учёта;
- отсутствие модуля аналитики на стороне продавца;
- жестко фиксированная структура данных;
- обладает сложным интерфейсом и ограниченными возможностями настройки.

Ярмарка Мастеров – российская торговая площадка для авторских изделий и художественных работ.

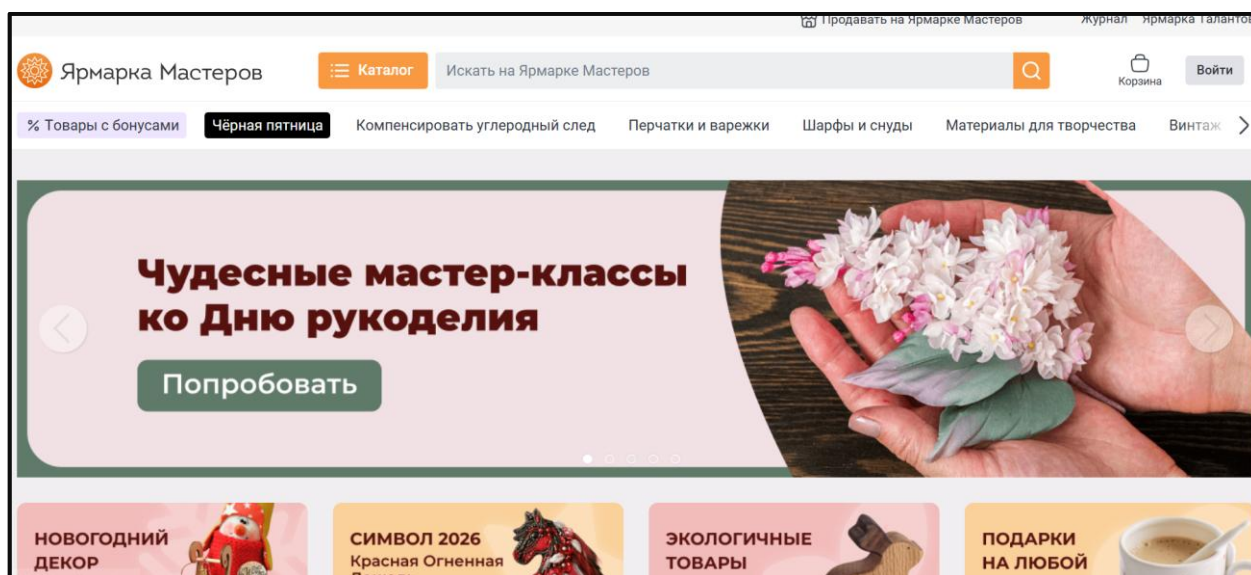


Рисунок 1.3 – Интерфейс Ярмарки Мастеров

Достоинства:

- наличие API с базовыми методами получения списка заказов и товаров;
- поддержка стандартных полей учёта: статус заказа, дата, стоимость, платёжный статус.

Недостатки:

- ограниченный состав данных в API;
- невозможность расширения структуры товара;
- нет возможности вести остатки, инвентаризацию или перемещения произведений;
- нет модуля аналитики.

Проведённый анализ показал, что большинство существующих решений ориентированы на глобальный рынок и не учитывают особенности локальных сообществ художников и индивидуальных продавцов. Также системы имеют закрытую архитектуру данных, не предоставляют полного доступа к информации о клиентах и транзакциях, не поддерживают расширяемую структуру учёта, специфичную для художественных произведений.

Учитывая выявленные в ходе обзора сильные и слабые стороны существующих аналогов, были сформулированы требования к разрабатываемой информационной системе.

1.2 Разработка требований к информационной системе

1.2.1 Бизнес-требования

БТ-1 Автоматизация процессов учёта и продажи произведений искусства.

БТ-2 Сокращение ручных операций и времени обработки заказов.

БТ-3 Повышение удобства взаимодействия между покупателем и продавцом.

БТ-4 Обеспечение прозрачности сделок и надёжности хранения данных.

БТ-5 Поддержка индивидуальных заказов (возможность покупателю заказывать работу под конкретные требования).

БТ-6 Формирование отчётов о продажах, доходах и активности пользователей.

БТ-7 Упрощение работы администратора при контроле качества данных и учёте операций.

1.2.2 Бизнес-правила

БП-1 Один пользователь может иметь только одну активную роль (администратор, покупатель или продавец).

БП-2 Работа может быть продана только один раз.

БП-3 Все платежи проходят через систему учёта и фиксируются в базе данных.

БП-4 Продавец может создавать не более 500 карточек товаров одновременно.

БП-5 Цена произведения искусства не может быть отрицательной.

БП-6 Отзыв может оставить только покупатель, получивший товар.

БП-7 Каждый индивидуальный заказ проходит этапы: создание запроса, предложение продавца, согласование, предоплата, выполнение, завершение.

БП-8 При превышении сроков выполнения индивидуального заказа система уведомляет обе стороны и администратора.

БП-9 Вся переписка и файлы по индивидуальным заказам сохраняются в базе данных.

БП-10 Администратор имеет право аннулировать заказ при нарушении правил платформы.

1.2.3 Пользовательские требования

ПТ-1 Администратор:

ПТ-1.1 управляет учётными записями пользователей (создает, удаляет, редактирует);

ПТ-1.2 назначает пользователям роли (покупатель, продавец, организатор);

ПТ-1.3 контролирует корректность данных;

ПТ-1.4 просматривает и выгружает отчёты по продажам, доходам и индивидуальным заказам в форматах CSV и PDF;

ПТ-1.5 модерирует отзывы и может скрывать их при нарушении правил платформы.

ПТ-2 Покупатель:

ПТ-2.1 просматривает каталог;

ПТ-2.2 фильтрует товары по цене, автору, типу искусства;

ПТ-2.3 оформляет заказ;

ПТ-2.4 оплачивает заказ и получает чек;

ПТ-2.5 создает запрос на индивидуальный заказ, указывая описание требований, желаемый бюджет, сроки и прилагая изображения-референсы;

ПТ-2.6 участвует в согласовании индивидуального заказа с продавцом через переписку;

ПТ-2.7 вносит предоплату по индивидуальному заказу после подтверждения от продавца;

ПТ-2.8 получает подтверждение заказа;

ПТ-2.9 просматривает историю своих покупок и оставляет отзыв.

ПТ-3 Продавец:

ПТ-3.1 управляет своими карточками товара: создание, редактирование, удаление карточки;

ПТ-3.2 указывает при создании товара следующую информацию: название, тип искусства, количество товара, цену, статус (в продаже / продана / снята с продажи / индивидуальный заказ), материалы;

ПТ-3.3 просматривает статистику продаж;

ПТ-3.4 принимает/отклоняет/переговаривает индивидуальные заказы;

ПТ-3.5 общается с покупателем в рамках индивидуального заказа через переписку;

ПТ-3.6 получает уведомления о новых заказах и изменениях статусов;

ПТ-3.7 отправляет заказ и подтверждает выполнение заказа в системе;

ПТ-3.8 отмечает завершение индивидуального заказа и предоставляет электронный акт выполненных работ.

1.2.4 Ограничения

О-1 Авторизация обязательна для всех действий, связанных с заказами или редактированием данных.

О-2 Хранение данных в системе – не менее трёх лет.

О-3 Заказ аннулируется, если не оплачен в течение 24 часов после подтверждения.

О-4 Один продавец может одновременно обрабатывать не более 20 индивидуальных заказов.

О-5 Максимальный срок выполнения индивидуального заказа – 90 календарных дней.

О-6 Все файлы, передаваемые в рамках индивидуальных заказов, не должны превышать 20 МБ.

2 Проектирование информационной системы

2.1 Выбор и обоснование архитектуры ИС и используемого ПО

Разрабатываемая информационная система построена по трехуровневой клиент-серверной архитектуре (рисунок 2.1). В неё входит:

- уровень представления (клиентское приложение);
- уровень бизнес-логики (сервер приложения);
- уровень данных (СУБД).

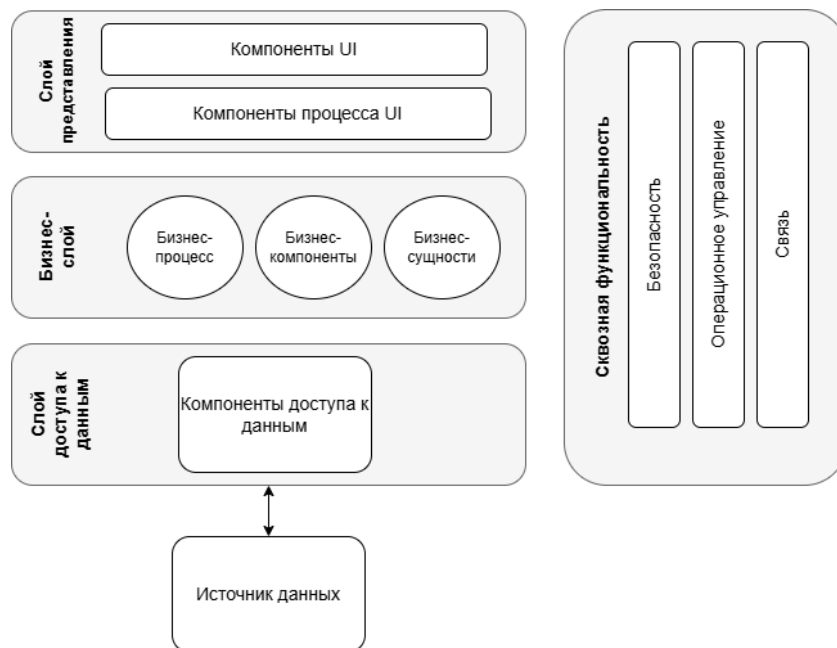


Рисунок 2.1 – Трехуровневая клиент-серверная архитектура

Описание компонентов технологии:

- а) основные технологии
 - 1) .NET Core / .NET [1] – платформа для разработки;
 - 2) ASP.NET Core [1] – веб-фреймворк;
- б) источник данных:
 - 1) PostgreSQL – система управления базами данных;
- в) слой доступа к данным (компоненты доступа к данным):
 - 1) Entity Framework Core – ORM для работы с БД;
 - 2) Npgsql – ADO.NET драйвер для PostgreSQL;
 - 3) папка Models с описанием форматов данных БД;
- г) бизнес слой:
 - 1) бизнес процесс реализован в логике файлов .cs;

- 2) бизнес компоненты реализованы классами C#;
 - 3) бизнес сущности реализованы в классах аккаунтов (Account), продуктов (Product), заказов (Purchase);
- д) слой представления:
- 1) компоненты UI реализованы HTML5/CSS3/JavaScript, Bootstrap;
 - 2) компоненты процесса UI реализованы на Razor Pages (.cshtml, .cshtml.cs);
- е) сквозная функциональность:
- 1) безопасность реализована на ASP.NET Core Identity – аутентификация и авторизация, HTTPS/SSL – шифрование трафика;
 - 2) операционное управление реализована доступом администратора к данным на всех таблицах БД;
 - 3) связь реализована Async/Await - асинхронная связь между всеми слоями.

Такая архитектура выбрана по следующим причинам:

- масштабируемость разрабатываемой информационной системы;
- поддержка многопользовательской работы;
- разделение прав доступа между уровнями;
- снижение нагрузки на устройстве клиента.

Роли уровней

Уровень предоставления отвечает за:

- отображение каталога произведений искусства;
- фильтрацию и поиск по типу искусства, автору, цене;
- формулирование запросов на индивидуальный заказ;
- отображение статусов заказов и историй покупок.

Уровень бизнес-логики реализует:

- авторизацию пользователя;
- управление бизнес-процессами: оформление заказа, подтверждение

индивидуального заказа, генерация отчётов.

Уровень данных обеспечивает:

- хранение и управление данными;
- целостность данных с помощью ограничений (PRIMARY KEY, FOREIGN KEY, CHECK);
- выполнение бизнес-логики через хранимые процедуры и триггеры.

Бизнес-логика распределена между уровнями следующим образом.

На уровне СУБД реализованы:

- а) ограничения CHECK для проверки корректности данных (SQL):
 - 1) формат: CHECK (email ~ '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\$'), CHECK (phone ~ '^[0-9]{11}\$');
 - 2) допустимые значения: CHECK (role_name IN ('seller', 'buyer', 'admin')), CHECK (status IN ('available', 'reserved', 'sold')), CHECK (method_delivery IN ('courier', 'pickup', 'mail')), CHECK (status_item IN ('pending', 'shipped', 'delivered', 'cancelled', 'accepted', 'in progress'));
 - 3) неотрицательные значения: CHECK (price > 0), CHECK (quantity_for_sale >= 0 AND quantity_for_sale <= 10000);
- б) ссылочная целостность, обеспечиваемая с помощью внешних ключей с политикой ON DELETE NO ACTION и ON UPDATE CASCADE.

На уровне приложения(ASP.NET Core[]) реализованы:

- а) управление ролями пользователей;
- б) бизнес-процессы, требующие сложных операций или взаимодействия с несколькими сущностями (например, создание индивидуального заказа).

На уровне интерфейса (HTML) реализованы:

- а) проверка вводимых данных прямо в браузере;
- б) динамические действия: фильтрация без перезагрузки страницы, открытие модульных окон, обновление статуса элементов;

в) реализация пользовательского интерфейса выполнена с использованием шаблона Razor Pages, который является частью фреймворка ASP.NET Core (этот подход позволяет создавать динамические веб-страницы, объединяя HTML-разметку с серверным кодом C#) [1].

В качестве СУБД выбрана PostgreSQL, так как она:

- поддерживает регулярные выражения в CHECK-ограничениях, что позволяет точно проверять ввод данных;
- обеспечивает высокую надёжность хранения данных и целостность транзакций;
- имеет открытую лицензию;
- позволяет реализовывать сложную бизнес-логику с помощью триггеров и процедур.

2.2 Объектно-ориентированное проектирование

При проектировании курсового проекта был выбран объектно-ориентированный подход, в рамках которых были разработаны UML-диаграммы.

Диаграмма вариантов использования [4] (рисунок 2.2) демонстрирует основные функции, доступные каждой роли. Покупатель может просматривать общий каталог товаров, фильтровать их по категориям, просматривать описание товара, выбирать товар из каталога и оформлять покупку – как стандартную, так и индивидуальную (на заказ). При оформлении заказа покупатель выбирает тип искусства и материал изделия, оплачивает заказ и получает чек и подтверждение от продавца. Зарегистрированный покупатель дополнительно может получить подтверждение от продавца.

Продавец, в свою очередь, может выставлять предмет на продажу, редактировать свои товары, указывать цену, добавлять описание и фото, обновлять статус товара, делать индивидуальный заказ, подтверждать заказ, отправлять товар и чек покупателю. Для выполнения индивидуального заказа продавец может оформить карточку нового товара, указать цену и материалы, а затем продать его или отменить заказ.

Администратор управляет пользователями и товарами: он может добавлять, редактировать и удалять пользователей и товары. Зарегистрированный администратор имеет расширенные права на управление системой.

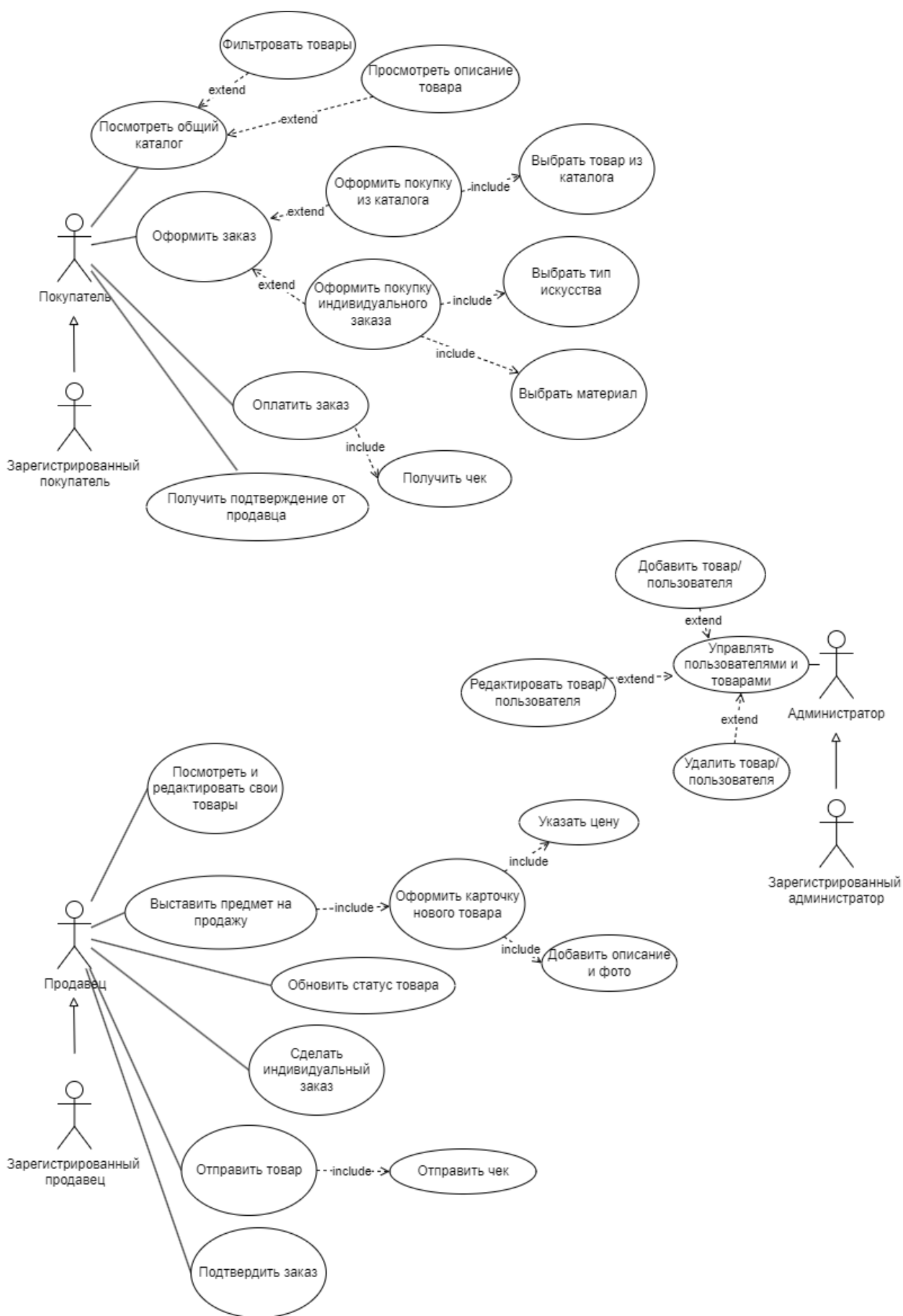


Рисунок 2.2 – Диаграмма вариантов использования

Диаграмма активности покупателя [4] (рисунок 2.3) описывает процесс оформление заказа. Процесс начинается с выбора покупателем действия: сделать индивидуальный заказ или выбрать товар из каталога. Если выбрано «из каталога», покупатель переходит к выбору конкретного товара из доступных позиций. Далее клиент переходит к оформлению заказа, и покупка автоматически считается подтверждённым. При выборе «индивидуального заказа» покупатель обсуждает в чате с продавцом товар, указывает тип изделия и материал, после чего система направляет запрос продавцу. Далее продавец создает индивидуальный товар, и только после этого покупатель может оплатить заказ.

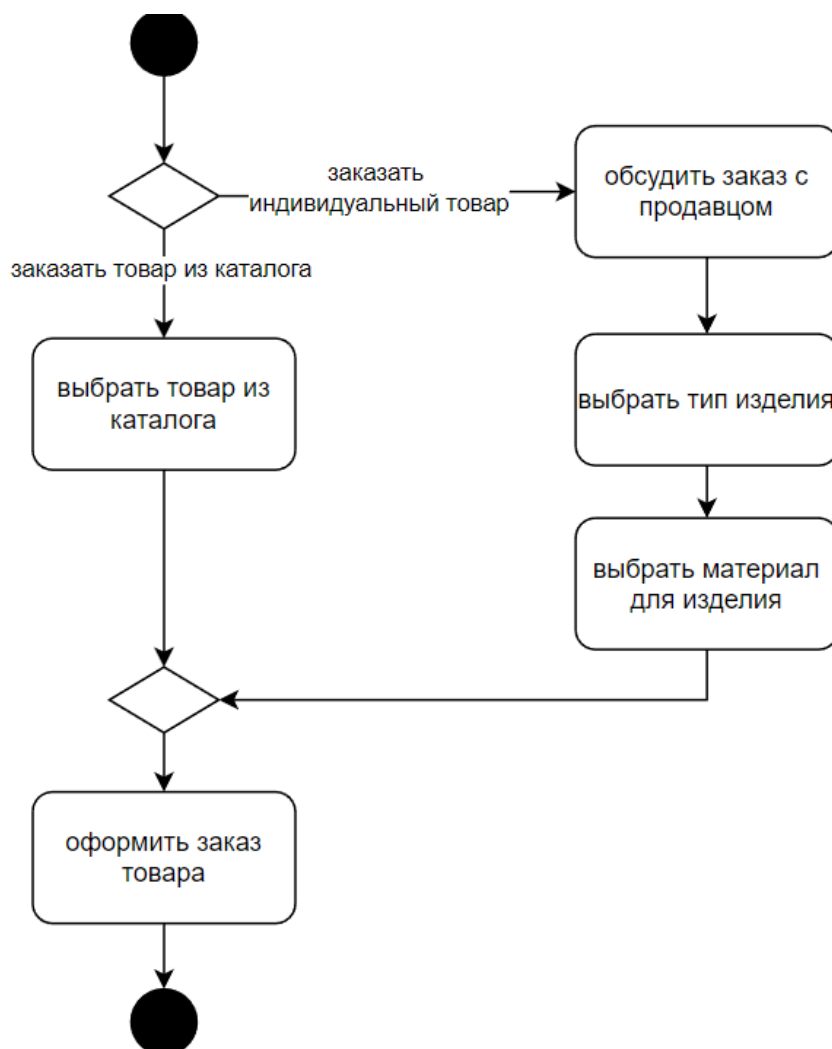


Рисунок 2.3 – Диаграммы активности покупателя

Диаграмма активности продавца [4] описывает сценарий создание товара для общего каталога (рисунок 2.4). Процесс начинается с того, что продавец добавляет описание товара, затем указывает его цену и сохраняет информацию в базе данных. После этого товар становится доступен для всех покупателей. При поступлении заказа на этот товар продавец подтверждает его, получает оплату и отправляет товар покупателю. Если заказ не подтвержден или оплата не поступила — продавец может отменить заказ.

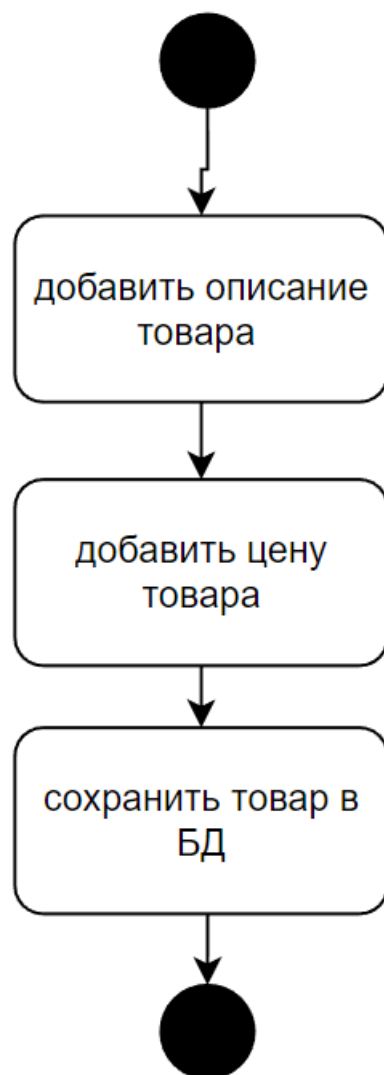


Рисунок 2.4 – Диаграммы активности продавца

Диаграмма класса [4] (рисунок 2.5) представляет собой объектную модель данных разрабатываемой информационной системы. Она визуализирует ключевые сущности предметной области, их атрибуты, типы данных и взаимосвязи, реализованные на уровне приложения.

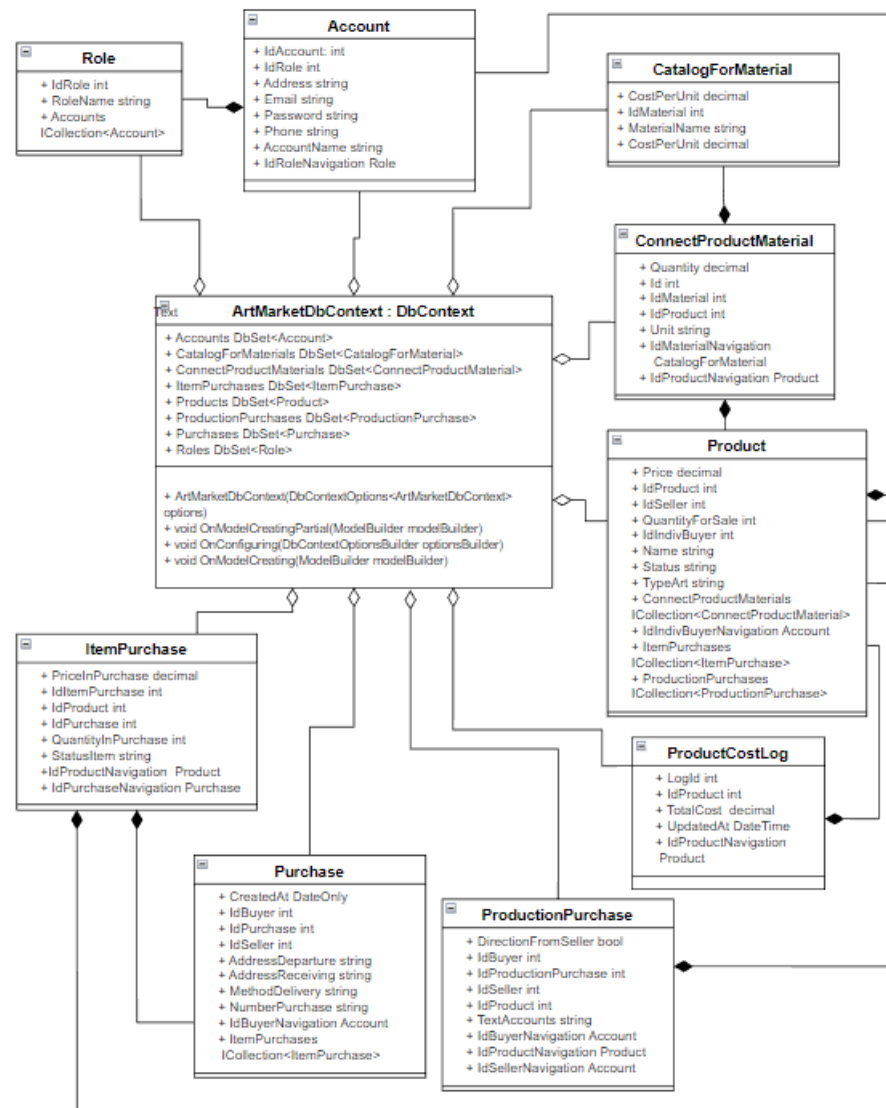


Рисунок 2.5 – Диаграмма класса для Models

Диаграмма классов для таблицы **Product** [4] (рисунок 2.6) отображает структуру, реализующую функционал управления сущностью Product через стандартный набор операций создания, чтения, редактирования, удаления. Центральным элементом является класс Product, который является сущностью базы данных. Он содержит все свойства, соответствующие полям таблицы в БД, включая проверки на уровне приложения. Основную логику обработки запросов инкапсулируют классов-моделей, каждый из которых отвечает за конкретный пользовательский сценарий.

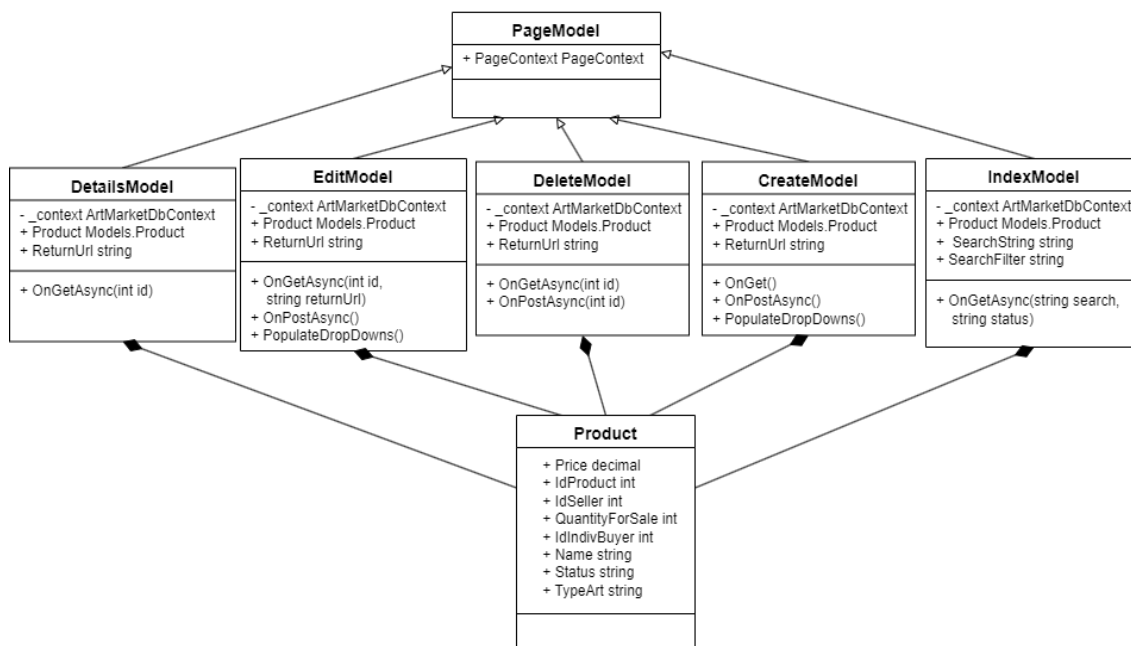


Рисунок 2.6 – Диаграмма класса для таблицы Product

Эти диаграммы позволяют четко определить границы ответственности каждой роли, последовательность действий и логику работы системы, что является основой для дальнейшей разработки архитектуры и реализации программного обеспечения.

2.3 Проектирование базы данных

На основе анализа требований системы были выделены следующие сущности:

- Account – аккаунты пользователей системы (продавцы, покупатели, администраторы);
- Role – роли пользователей;
- Product – предметы искусства, выставляемые на продажу;
- Purchase – заказы на покупку товаров;
- Item_purchase – позиции в заказах;
- Production_purchase – заказы на изготовление индивидуальных предметов искусства;
- Catalog_for_material – справочник используемых материалов;
- Connect_product-material – связь товара с материалами, из которых он состоит.

База данных приведена к третьей нормальной форме (3НФ). Это означает, что в ней отсутствуют транзитивные зависимости, то есть ни одно неключевое поле не зависит от другого неключевого поля через ключевое.

Обоснование выполнения условий 3НФ:

- условие 1: база данных находится в первой нормальной форме (1НФ). Все таблицы содержат только неделимые значения. Например, account_name, email, phone содержат одиночные строки, а не списки или составные данные;
- условие 2: база данных находится во второй нормальной форме (2НФ). Все неключевые атрибуты полностью зависят от первичного ключа, а не его части. Поскольку все таблицы имеют первичный ключ, то автоматически выполняется условие 2НФ;
- условие 3: база данных находится в третьей нормальной форме (3НФ). Отсутствуют транзитивные зависимости. Это достигнуто за счет вынесения зависимых атрибутов в отдельные таблицы.

Пример 1: связь Account и Role. Поле role_name в таблице Account зависит от ID_role, а ID_role – от первичного ключа ID_account. Это транзитивная зависимость. Чтобы ее устранить, создана отдельная таблица Role, где role_name зависит только от своего первичного ключа ID_role. Теперь в таблице Account хранится только ID_role, который ссылается на Role.

Пример 2: связь Product и Material. Информация о материале (его название и цена) зависит от ID_material, а ID_material – от ID_product через связь в таблице Connect_product_material. Это также транзитивная зависимость. Для ее устранения создана отдельная таблица Catalog_for_material, где material_name и cost_per_unit зависят только от ID_material.

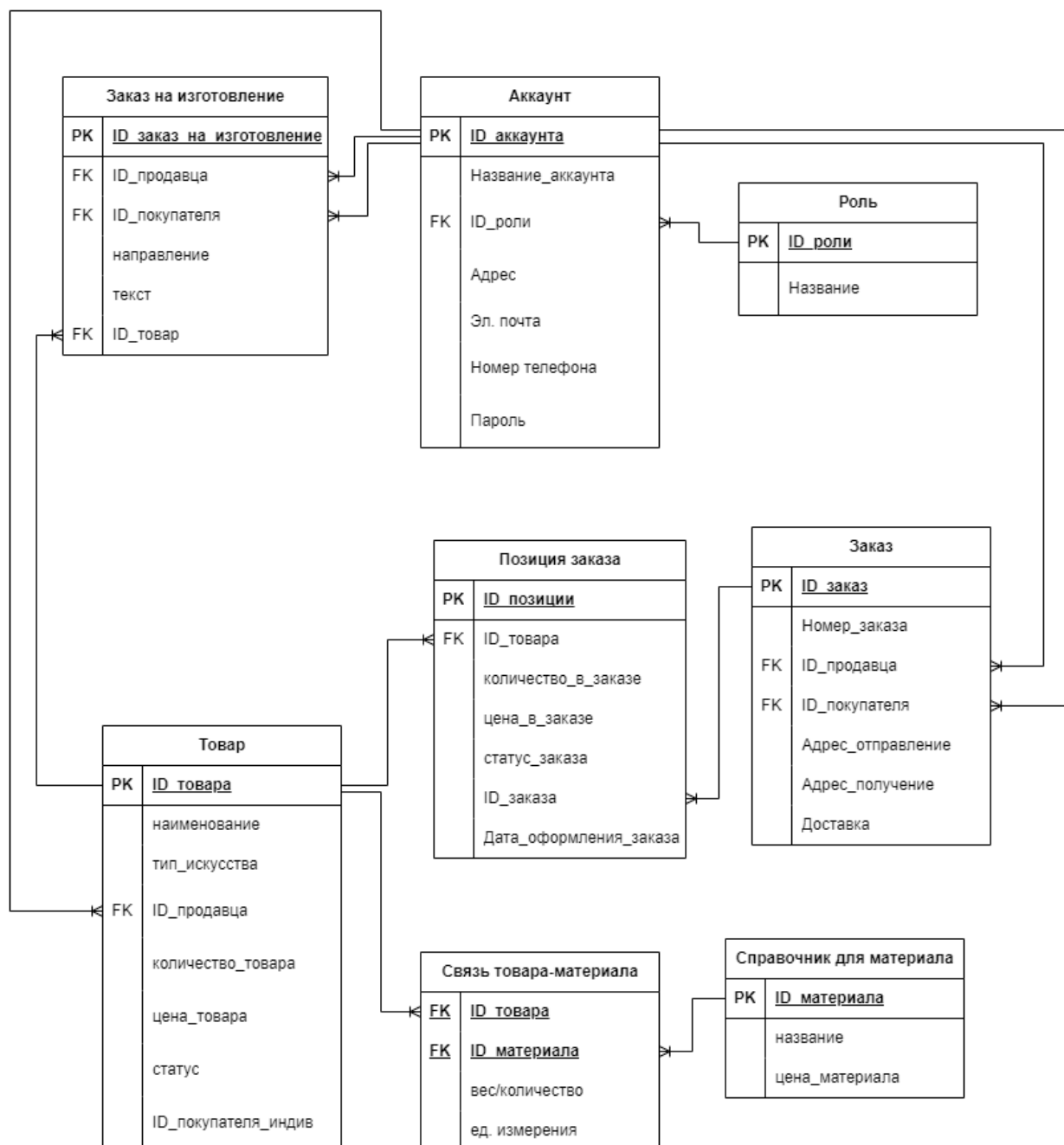


Рисунок 2.7 – Логическая модель базы данных

Ограничения целостности

Для обеспечения корректности и целостности данных в базе данных реализованы следующие ограничения.

Обязательные поля (NOT NULL):

- а) account_name, email, phone, password, ID_role в таблице Account;
- б) name, type_art, ID_saller, price, status в таблице Product;
- в) number_purchase, ID_seller, ID_buyer, method_delivery в таблице Purchase;

Проверка значений (CHECK):

- а) роль пользователя: role_name IN ('seller', 'buyer', 'admin');
- б) статус товара: status IN ('available', 'reserved', 'sold');
- в) статус позиции заказа: status_item IN ('pending', 'shipped', 'delivered', 'cancelled', 'accepted', 'in progress');
- г) метод доставки: method_delivery IN ('courier', 'pickup', 'mail');
- д) количество товара: $0 \leq \text{quantity_for_sale} \leq 10000$;
- е) количество в заказе: $0 < \text{quantity_in_purchase} \leq 100$;
- ж) форматы данных:
 - 1) email соответствует регулярному выражению ($^{\wedge}[A-Za-z0-9._%+-][+@][A-Za-z0-9.-]+\.[A-Za-z]{2,}\$$);
 - 2) phone соответствует 11-значному числу ($^{\wedge}[0-9]{9}\$$);
 - 3) account_name, name, type_art, material_name допускают только безопасные символы.

Уникальные значения (UNIQUE):

- а) email в таблице Account;
- б) комбинации (ID_product, ID_material) в таблице Connect_product-material.
- в) Внешние ключи (FOREIGN KEY):
- г) при удалении роли (Role) — запрещено (ON DELETE NO ACTION), чтобы не нарушить целостность аккаунтов;
- д) при обновлении роли – каскадно (ON UPDATE CASCADE);
- е) при удалении аккаунта – запрещено (ON DELETE NO ACTION), поскольку это может повлиять на заказы и товары;
- ж) при обновлении ID_account – каскадно (ON UPDATE CASCADE);
- з) связи между Product, Purchase, Item_purchase, Production_purchase, Catalog_for_material реализованы аналогично: ON DELETE NO ACTION, ON UPDATE CASCADE.

Обоснование выбора индексов:

а) индексы по внешним ключам (ID_role, ID_saller, ID_buyer, ID_product, ID_purchase, ID_material) – ускоряют соединения (JOIN) и проверку ссылочной целостности;

б) составные индексы – например, по (ID_saller, status) в Product – оптимизируют запросы продавцов о своих товарах;

в) индексы по статусам (status, status_item) – ускоряют фильтрацию текущих и завершённых операций;

г) индексы по датам – не используются напрямую, но могут быть добавлены в будущем при расширении функционала (например, для аналитики по периодам).

Для хранения паролей выбран тип VARCHAR(255), что обеспечивает совместимость с современными алгоритмами хеширования (например, bcrypt), гибкость при обновлении методов шифрования и защиту от атак по rainbow-таблицам.

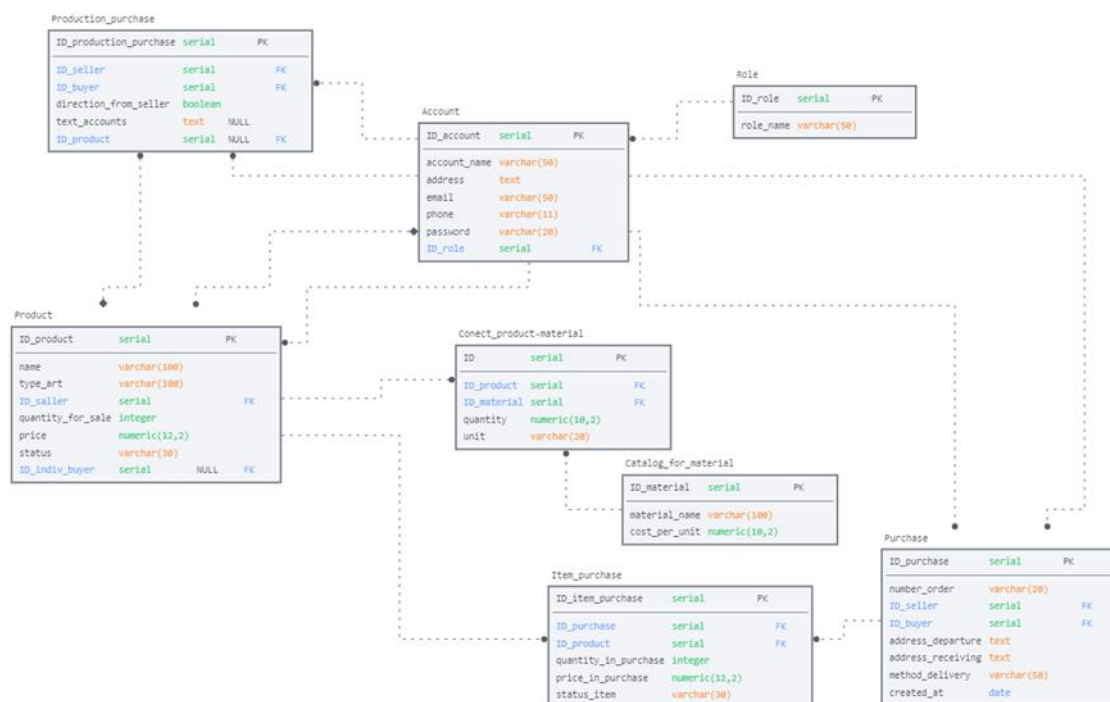


Рисунок 2.8 – Физическая модель базы данных (схема таблиц с типами полей, ключами и ограничениями)

3 Разработка информационной системы

3.1 Разработка бизнес-логики на уровне СУБД

Реализованные функции.

(1) Статистика продаж за введённый период по типам искусства.

Функция `sales_by_art_type(start_date DATE, end_date DATE)` реализует формирование отчётов о продажах.

Она принимает диапазон дат и возвращает статистику по каждому типу искусства: общее количество проданных единиц и выручку. В расчёт включаются только позиции заказов со статусами 'accepted' или 'delivered', что гарантирует учёт только завершённых сделок.

(2) Создание индивидуального товара по заказу на изготовление

Процедура `create_custom_product(p_order_id, p_name, p_type_art, p_price)` реализует этапы индивидуального заказа.

Она принимает ID заказа на изготовление (`Production_purchase`) и характеристики нового изделия, извлекает ID продавца и покупателя из заказа, затем создаёт запись в таблице `Product` со статусом 'reserved' и связью с конкретным покупателем через поле `ID_indiv_buyer`. Это позволяет индивидуальному изделию появиться в каталоге только для его заказчика.

(3) Получение списка материалов для товара с расчётом стоимости

Функция `get_materials_for_product(p_product_id)` поддерживает прозрачность себестоимости и реализует надёжность и прозрачность данных.

Она возвращает для указанного товара: наименование материала, количество, единицу измерения и итоговую стоимость (количество * цена за единицу). Это может использоваться продавцом при формировании цены или администратором при аудите.

Реализованные триггеры

(1) Списание товара при подтверждении заказа

Триггер `trg_deduct_on_accept` срабатывает при изменении статуса позиции заказа (`Item_purchase`) на 'accepted'.

Он проверяет, является ли товар индивидуальным (через поле `ID_indiv_buyer`). Для массовых товаров уменьшает `quantity_for_sale` и, при достижении нуля, переводит статус в 'sold'. Для индивидуальных – списание не производится (они создаются под заказ и не хранятся на складе).

(2) Проверка наличия товара при добавлении в заказ

Триггер `trg_check_stock_insert` запускается при вставке новой позиции в `Item_purchase`.

Он аналогично различает массовые и индивидуальные товары. Для первых – проверяет, достаточно ли товара на складе (`quantity_for_sale >= quantity_in_purchase`). При недостатке – генерирует исключение, предотвращая некорректное оформление заказа.

(3) Логирование себестоимости при изменении состава изделия

Триггер `trg_log_cost_on_material_change` срабатывает при любом изменении (INSERT, UPDATE, DELETE) в таблице `Connect_product_material`.

Он пересчитывает общую стоимость материалов для соответствующего товара и сохраняет результат в служебную таблицу `Product_cost_log` с отметкой времени. Это позволяет отслеживать историю изменения себестоимости.

3.2 Разработка бизнес-логики на уровне приложения

Приложение реализовано на платформе ASP.NET Core 9.0 [1] с использованием шаблона Razor Pages [1]. Взаимодействие с базой данных PostgreSQL осуществляется через Entity Framework Core [2] с провайдером Npgsql.

Архитектура приложения следует принципам Razor Pages: логика обработки запросов сосредоточена в файлах кода страниц (.cshtml.cs), а отображение – в соответствующих .cshtml-файлах.

Основные сценарии работы реализованы следующим образом:

- авторизация и управление доступом. Приложение поддерживает три роли: администратор, продавец и покупатель. Доступ к работе приложения ограничен на основе роли пользователя, которая хранится в таблице Account. При входе система определяет роль и использует атрибуты для защиты страниц.

- работа с данными через Entity Framework Core. Все операции чтения, создания, обновления и удаления (CRUD) выполняются с помощью объектов DbContext и методов Add, Update, Remove, SaveChangesAsync. Это стандартный подход, предоставляемый EF Core.

- интеграция с бизнес-логикой СУБД. Основные бизнес-процессы, такие как создание индивидуального товара, реализованы в виде хранимых процедур в PostgreSQL. На уровне приложения эти процедуры вызываются с помощью методов ExecuteSqlRaw или ExecuteSqlInterpolated.

- валидация данных. Валидация входных данных на стороне сервера реализована с помощью атрибутов DataAnnotations в моделях. Эти атрибуты ([Required], [StringLength], [RegularExpression]) автоматически применяются при отправке формы. В коде страницы PageModel используется проверка if (!ModelState.IsValid) return Page();, что гарантирует корректность данных перед их сохранением в БД.

– обработка ошибок и безопасность. Все критические операции (создание, изменение, удаление) защищены от повторной отправки формы (Post/Redirect/Get).

3.3 Разработка интерфейса ИС

Пользовательский интерфейс разработан с использованием Razor Pages в ASP.NET Core 9. Дизайн ориентирован на простоту, функциональность и соответствие ролевой модели: администратор, продавец и покупатель видят только те элементы интерфейса, которые соответствуют их правам доступа.

При первом посещении сайта пользователь автоматически направляется на страницу «Каталог товаров» (рисунок 3.1), где он может свободно просматривать все доступные произведения искусства. Для выполнения любых действий, связанных с заказами, редактированием данных или администрированием, обязательна авторизация.

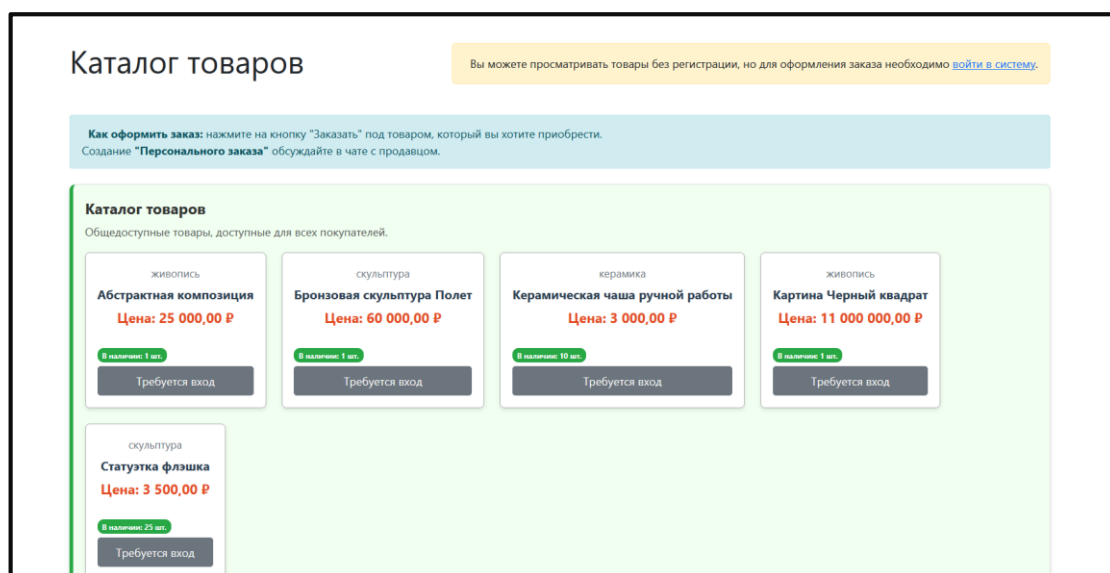


Рисунок 3.1 – Страница каталога товаров для анонимного пользователя

После успешной авторизации (рисунок 3.2) пользователь получает доступ к расширенному функционалу в зависимости от своей роли (покупатель, продавец, администратор). На странице авторизации пользователь вводит название аккаунта/логин и пароль, после чего система перенаправляет его на соответствующую страницу.

Вход в систему

Имя или Email

Иван, ivan@example.com

Введите имя пользователя или email

Пароль

Показать

Войти

[Нет аккаунта? Зарегистрироваться](#)

Рисунок 3.2 – Страница авторизации

Для пользователя в роли покупателя основной страницей становится обновленный «Каталог товаров» (рисунок 3.3). Теперь кнопка «Заказать» активна и позволяет перейти к оформлению заказа (рисунок 3.4). Пользователь может выбрать количество товара, способ доставки и ввести адрес получения. После подтверждения заказа система пишет, что заказ успешно оформлен.

Каталог товаров

Как оформить заказ: нажмите на кнопку "Заказать" под товаром, который вы хотите приобрести.
Создание "Персонального заказа" обсуждайте в чате с продавцом.

Ваши персональные товары
Товары, созданные специально для вас по индивидуальному заказу.

керамика
Персональная ваза для Ивана
Цена: 4 500,00 Р
Заказать

Каталог товаров
Общедоступные товары, доступные для всех покупателей.

живопись Абстрактная композиция Цена: 25 000,00 Р Заказать	скульптура Бронзовая скульптура Полет Цена: 60 000,00 Р Заказать	керамика Керамическая чаша ручной работы Цена: 3 000,00 Р Заказать	живопись Картина Черный квадрат Цена: 11 000 000,00 Р Заказать
скульптура Статуэтка флешка Цена: 3 500,00 Р Заказать			

Рисунок 3.3 – Страница каталога товаров для авторизованного покупателя

Оформление заказа: Статуэтка флэшка

Количество товара: Доступно: 25 шт.

1

Максимально можно заказать: 25 шт.

Способ доставки

Выберите способ доставки

Адрес получения товара

Укажите полный адрес для доставки

Укажите полный адрес для доставки

Комментарий к заказу (опционально)

Максимум 500 символов

0/500 символов

Итоговая информация

Товар: Статуэтка флэшка

Цена за единицу: 3 500,00 Р

Количество: 1 шт.

Итоговая стоимость: 3 500,00 Р

Стоимость доставки будет уточнена продавцом после подтверждения заказа

Отмена
Подтвердить заказ

Рисунок 3.4 – Страница оформлению заказа

Для пользователя в роли продавца главной страницей является страница управления товарами (рисунок 3.5). Здесь он видит список всех своих выставленных на продажу произведений и заказы. Для каждого товара доступны действия: «Просмотреть», «Редактировать» и «Удалить». Так же есть кнопка «Добавить товар», которая открывает форму для добавления как нового товар для каталога, так и индивидуальный товар (рисунок 3.6).

Панель продавца: Арт Галерея Добавить товар

Мои товары				
Название	Тип	Цена	Статус	Действия
Абстрактная композиция	живопись	25000,00	В продаже	Просмотреть Редактировать Удалить
Бронзовая скульптура Полет	скульптура	60000,00	В продаже	Просмотреть Редактировать Удалить
Портрет Анны в интерьере	живопись	35000,00	Зарезервирован Персональный заказ	Просмотреть Редактировать Удалить
Ваза малахитовая фарфоровая	фарфор	5000,00	В продаже Персональный заказ	Просмотреть Редактировать Удалить
Ваза песчаная	скульптура	100,00	В продаже Персональный заказ	Просмотреть Редактировать Удалить

ЗАКАЗЫ 2

Заказ #PO-2025-001 1 шт.

Абстрактная композиция

Цена за единицу
25 000,00 Р

Сумма заказа
25 000,00 Р

Адрес доставки
г. Москва, ул. Тверская 15

Дата: 20.11.2025 ID товара: 1

Заказ #PO-2025-002 1 шт.

Портрет Анны в интерьере

Цена за единицу
35 000,00 Р

Сумма заказа
35 000,00 Р

Адрес доставки
г. Москва, ул. Тверская 15

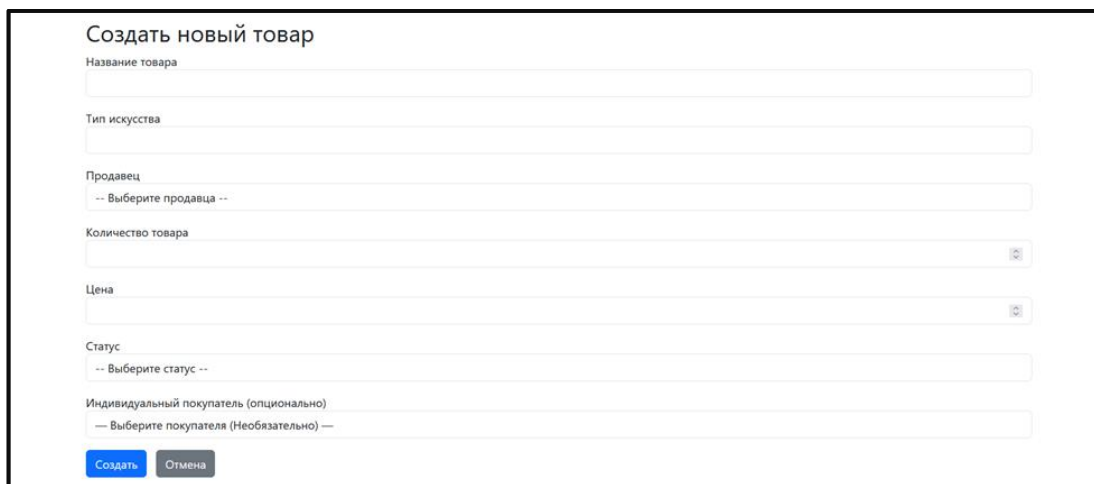
Дата: 22.11.2025 ID товара: 4

Всего заказов: 2

Рисунок 3.5 – Страница управления товарами (роль «продавец»)

Форма создания (рисунок 3.7) или редактирования товара (рисунок 3.7) содержит поля для ввода названия, типа искусства, количества, цены, статуса

и опционально – выбора покупателя для индивидуального заказа. После сохранения данные сразу же обновляются в базе данных.



Создать новый товар

Название товара

Тип искусства

Продавец

-- Выберите продавца --

Количество товара

Цена

Статус

-- Выберите статус --

Индивидуальный покупатель (опционально)

— Выберите покупателя (Необязательно) —

Создать Отмена

Рисунок 3.7 – Страница добавления товара



Редактировать товар

Название товара

Керамическая чаша ручной работы

Тип искусства

керамика

Продавец

2

Количество товара

10

Цена

3000.00

Статус

В наличии

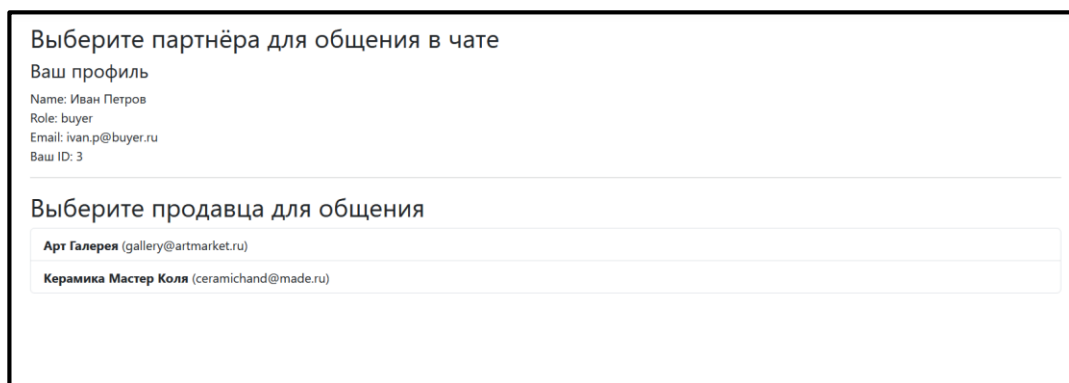
Индивидуальный покупатель (опционально)

— Не выбран —

Сохранить Отмена

Рисунок 3.8 – Страница редактирования товара

Продавец и покупатель также могут общаться через чат. На странице «Чат» пользователь видит список чатов (рисунок 3.9), где могут клиент с продавцом могут обсудить индивидуальный товар (рисунок 3.10).



Выберите партнёра для общения в чате

Ваш профиль

Name: Иван Петров

Role: buyer

Email: ivan.p@buyer.ru

Ваш ID: 3

Выберите продавца для общения

Арт Галерея (gallery@artmarket.ru)

Керамика Мастер Коля (ceramichand@made.ru)

Рисунок 3.9 – Страница списка чатов

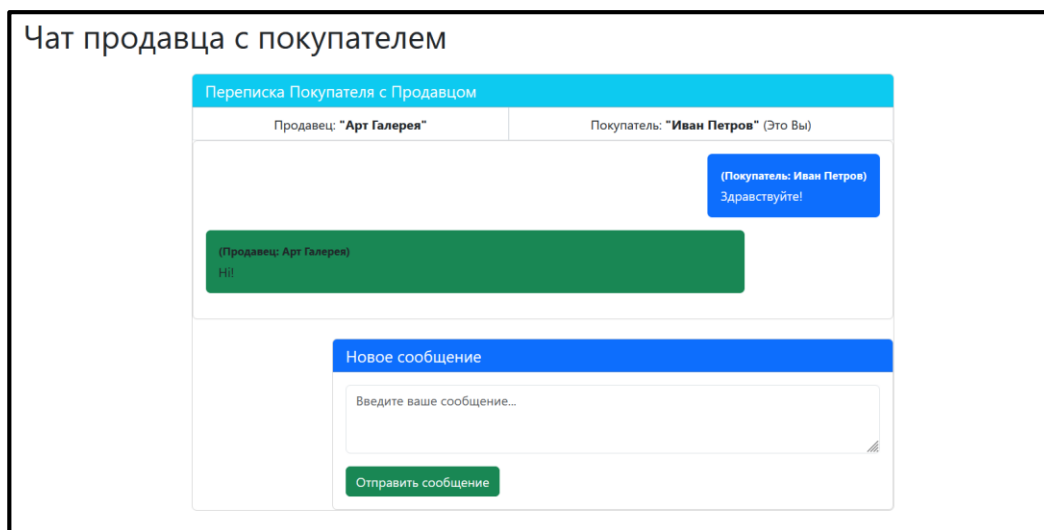


Рисунок 3.10 – Страница чата

Для пользователя в роли администратора доступна страница управления данными (рисунок 3.10). Она предоставляет собой быстрый доступ к разделам. Администратор может просматривать, редактировать и удалять записи, а также назначать или изменять роли пользователей. Эта страница обеспечивает полный контроль над пользовательской базой.

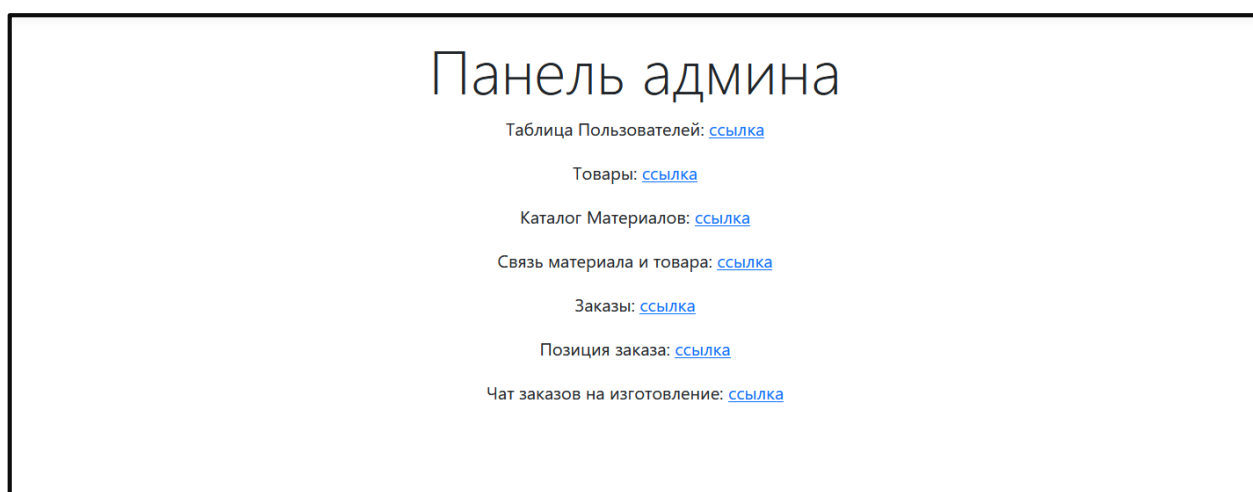


Рисунок 3.11 – Страница управления данными (роль «администратор»)

Для каждой основной сущности созданы отдельные CRUD-страницы (возьмем в пример Account):

- Index – список с возможностью поиска и постраничной навигации (рисунок 3.12);
- Create – форма создания новой записи с валидацией полей (рисунок 3.13);

- Edit – форма редактирования (рисунок 3.14);
- Details – просмотр полной информации (рисунок 3.15);
- Delete – подтверждение удаления (рисунок 3.16).

Пользователи

+ Создать нового пользователя

Поиск по имени, email или телефону

Все роли

Фильтровать

Сброс

Название аккаунта	Адрес	Email	Телефон	Роль	Действия
Иван Петров	г. Казань, ул. Баумана 5	ivan.p@buyer.ru	79345678901	buyer	Изменить Подробнее Удалить
Анна Сидорова	г. Екатеринбург, пр. Ленина 22	anna.s@client.ru	79456789012	buyer	Изменить Подробнее Удалить
Арт Галерея	г. Москва, ул. Тверская 111	gallery@artmarket.ru	79123456789	seller	Изменить Подробнее Удалить
Керамика Мастер Коля	г. Санкт-Петербург, Невский пр. 100	ceramichand@made.ru	79234567811	seller	Изменить Подробнее Удалить
АдмиНОДО	Нету-Р	son@gmail.com	89123310668	admin	Изменить Подробнее Удалить
Admin2	по(	chit@gmail.com	89123310000	admin	Изменить Подробнее Удалить

Рисунок 3.12 – Страница «Пользователь»

Создать нового пользователя

Имя пользователя

Адрес

Email

Телефон

Пароль

Показать

Роль
-- Выберите роль --

Создать

Отмена

Рисунок 3.13 – Страница создания пользователя

Редактировать пользователя

Имя пользователя
Иван Петров

Адрес
г. Казань, ул. Баумана 5

Email
ivan.p@buyer.ru

Телефон
79345678901

Пароль

••••••••

Показать

Роль
buyer

Сохранить

Отмена

Рисунок 3.14 – Страница редактирования пользователя

Детали пользователя	
Имя пользователя	Иван Петров
Адрес	г. Казань, ул. Баумана 5
Email	ivan.p@buyer.ru
Телефон	79345678901
Роль	buyer

[Изменить](#) [Назад к списку](#)

Рисунок 3.15 – Страница деталей пользователя

Удалить пользователя	
Вы уверены, что хотите удалить этого пользователя?	
Имя пользователя	Иван Петров
Адрес	г. Казань, ул. Баумана 5
Email	ivan.p@buyer.ru
Телефон	79345678901
Роль	buyer

[Удалить](#) [Отмена](#)

Рисунок 3.16 – Страница удаления пользователя

Доступ к этим страницам ограничен по ролям:

- покупатель может просматривать каталог и свои заказы, заказать товар;
- продавец может управлять своими товарами и заказами;
- администратор имеет полный доступ ко всем разделам, включая отчёты.

Таким образом, разработанный интерфейс системы обеспечивает четкое разделение прав доступа, гибкость в работе с данными и высокий уровень удобства для всех участников. Все страницы реализованы с использованием Razor Pages, что гарантирует стабильность, безопасность и возможность дальнейшего масштабирования системы. Интерфейс протестирован в браузере Firefox корректно отображается.

Заключение

В ходе выполнения курсового проекта была успешно разработана информационная система для учёта продаж предметов искусства, полностью соответствующая всем заявленным требованиям.

Были достигнуты все поставленные цели и решены задачи:

Проведён подробный анализ аналогов (Etsy, Saatchi Art, Ярмарка Мастеров), выявлены их недостатки: закрытая архитектура, отсутствие аналитики, невозможность локального развёртывания.

Сформулированы бизнес-логика и пользовательские требования, включая поддержку индивидуальных заказов, фильтрацию каталога и генерацию отчётов.

Спроектирована реляционная база данных в третьей нормальной форме с полной системой ограничений целостности (PK, FK, CHECK, UNIQUE).

Реализована многоуровневая бизнес-логика:

- на уровне СУБД – через функции, процедуры и триггеры;
- на уровне приложения – через ASP.NET Core и Entity Framework Core.

Разработан интуитивно понятный адаптивный веб-интерфейс.

Разработанная система выгодно отличается от существующих аналогов своей открытой архитектурой, гибкостью учёта индивидуальных заказов и расширенной аналитикой продаж.

Библиографический список

1. Microsoft Learn. Razor Pages: Getting Started [Электронный ресурс]: учебник/Microsoft Corporation. – Электрон. дан. – Redmond, Wa: Microsoft, 2025. – URL: https://learn.microsoft.com/ru-ru/aspnet/core/tutorials/razor-pages/razor-pages-start?spm=a2ty_o01.29997173.0.0.4c3a5171SATKUR&view=aspnetcore-7.0&tabs=visual-studio (2025, December 1).
2. Microsoft Learn. Entity Framework Core with Razor Pages [Электронный ресурс]: учебник/Microsoft Corporation. – Электрон. дан. – Redmond, WA: Microsoft, 2025. – URL: https://learn.microsoft.com/ru-ru/aspnet/core/data/ef-rp/intro?spm=a2ty_o01.29997173.0.0.4c3a5171SATKUR&view=aspnetcore-7.0&tabs=visual-studio (2025, December 1).
3. Microsoft Learn. Validation in Razor Pages [Электронный ресурс]: учебник/Microsoft Corporation. – Электрон. дан. – Redmond, WA: Microsoft, 2025. – URL: <https://learn.microsoft.com/ru-ru/aspnet/core/tutorials/razor-pages/validation?view=aspnetcore-7.0&tabs=visual-studio> (2025, December 1).
4. Гарсиа-Молина, Г. Системы баз данных: полный курс / Г. Гарсиа-Молина, Д. Ульман, Д. Уидом. – Москва: Вильямс, 2022. – 1216 с.
5. Алексанян Г. В. Веб-программирование на платформе .NET: от основ к практике / Г. В. Алексанян, А. С. Кучеренко. – Москва: ДМК Пресс, 2024. – 384 с.
6. Коннолли, Т. Базы данных: проектирование, реализация и сопровождение. Теория и практика / Т. Коннолли, К. Бегг. – 6-е изд. – Москва: Вильямс, 2023. – 1376 с.
7. Кузнецов, С. Д. Базы данных: модель, разработка, реализация / С. Д. Кузнецов. – Санкт-Петербург: Питер, 2022. – 832 с.
8. Цельщиков, А. А. Проектирование информационных систем: от требований к архитектуре / А. А. Цельщиков. – Москва: ДМК Пресс, 2023. – 320 с.

9. Фаулер, М. Архитектура корпоративных программных приложений / М. Фаулер; пер. с англ. – Санкт-Петербург: Питер, 2023. – 544 с.
10. Алексаян, Г. В. Веб-программирование на платформе .NET: от основ к практике / Г. В. Алексаян, А. С. Кучеренко. – Москва: ДМК Пресс, 2024. – 384 с.
11. Штальберг, Е. Разработка веб-приложений с использованием ASP.NET Core и PostgreSQL / Е. Штальберг. – Москва: БХВ-Петербург, 2024. – 368 с.

Приложения

Приложение А. Листинг программы

База данных

```
CREATE TABLE Role
(
  ID_role serial NOT NULL, PRIMARY KEY (ID_role),
  role_name varchar(50) NOT NULL,
  CHECK (role_name IN ('seller', 'buyer', 'admin'))
);
--Аккаунт
CREATE TABLE Account
(
  ID_account serial NOT NULL, PRIMARY KEY ( ID_account ),
  account_name varchar(50) NOT NULL,
  CHECK (account_name ~ '^[A-Za-zA-Яа-я0-9\s\_-]+$'),
  address text NOT NULL,
  email varchar(50) NOT NULL,
  CHECK (email ~ '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'),
  phone varchar(11) NOT NULL,
  CHECK (phone ~ '^[0-9]{11}$'),
  password varchar(20) NOT NULL,
  ID_role integer NOT NULL,
  FOREIGN KEY ( ID_role ) REFERENCES Role ( ID_role )
    ON DELETE NO ACTION
    ON UPDATE CASCADE
);
--Товар
CREATE TABLE Product
(
  ID_product serial NOT NULL, PRIMARY KEY ( ID_product ),
  name varchar(100) NOT NULL,
  CHECK (name ~ '^[A-Za-zA-Яа-я0-9\s\-.!~?]+$'),
  type_art varchar(100) NOT NULL,
  CHECK (type_art ~ '^[A-Za-zA-Яа-я\s]+$'),
  ID_seller integer NOT NULL,
  quantity_for_sale integer NOT NULL,
  CHECK (quantity_for_sale >= 0 AND quantity_for_sale <= 10000),
  price numeric(12,2) NOT NULL,
  status varchar(30) NOT NULL,
  CHECK (status IN ('available', 'reserved', 'sold')),
  ID_indiv_buyer integer NULL,
  FOREIGN KEY ( ID_seller ) REFERENCES Account ( ID_account )
    ON DELETE NO ACTION
    ON UPDATE CASCADE,
  FOREIGN KEY ( ID_indiv_buyer ) REFERENCES Account ( ID_account )
    ON DELETE NO ACTION
    ON UPDATE CASCADE
);
--Заказ на изготовление
CREATE TABLE Production_purchase
(
  ID_production_purchase serial NOT NULL, PRIMARY KEY ( ID_production_purchase ),
  ID_seller integer NOT NULL,
  ID_buyer integer NOT NULL,
  direction_from_seller boolean NOT NULL,
  text_accounts text NULL,
  ID_product integer NULL,
  FOREIGN KEY ( ID_seller ) REFERENCES Account ( ID_account )
    ON DELETE NO ACTION
```



```

        ON UPDATE CASCADE,
FOREIGN KEY ( ID_buyer ) REFERENCES Account ( ID_account )
        ON DELETE NO ACTION
        ON UPDATE CASCADE,
FOREIGN KEY ( ID_product ) REFERENCES Product ( ID_product )
        ON DELETE NO ACTION
        ON UPDATE CASCADE,
CHECK (ID_seller != ID_buyer)
);
-- Материал
CREATE TABLE Catalog_for_material
(
    ID_material serial NOT NULL, PRIMARY KEY ( ID_material ),
    material_name    varchar(100) NOT NULL,
    cost_per_unit NUMERIC(10,2) NOT NULL,
    CHECK (material_name ~ '^[A-Za-zA-Яа-я0-9\s\~\-\+\$')',
    CHECK (cost_per_unit >= 0)
);
-- Связь товар-материал
CREATE TABLE Connect_product_material
(
    ID    serial NOT NULL, PRIMARY KEY ( ID ),
    ID_product integer NOT NULL,
    ID_material integer NOT NULL,
    quantity NUMERIC(10,3) NOT NULL,
    unit    VARCHAR(20) NOT NULL,
    FOREIGN KEY ( ID_product ) REFERENCES Product ( ID_product )
        ON DELETE NO ACTION
        ON UPDATE CASCADE,
    FOREIGN KEY ( ID_material ) REFERENCES Catalog_for_material ( ID_material )
        ON DELETE NO ACTION
        ON UPDATE CASCADE
);
--Заказ
CREATE TABLE Purchase
(
    ID_purchase    serial NOT NULL, PRIMARY KEY ( ID_purchase ),
    number_purchase varchar(20) NOT NULL,
    ID_seller    integer NOT NULL,
    ID_buyer    integer NOT NULL,
    address_departure text NOT NULL,
    address_receiving text NOT NULL,
    method_delivery varchar(50) NOT NULL,
    created_at    DATE NOT NULL,
    CHECK (method_delivery IN ('courier', 'pickup', 'mail')),
    FOREIGN KEY ( ID_seller ) REFERENCES Account ( ID_account )
        ON DELETE NO ACTION
        ON UPDATE CASCADE,
    FOREIGN KEY ( ID_buyer ) REFERENCES Account ( ID_account )
        ON DELETE NO ACTION
        ON UPDATE CASCADE
);
--Позиция заказа
CREATE TABLE Item_purchase
(
    ID_item_purchase serial NOT NULL, PRIMARY KEY ( ID_item_purchase ),
    ID_purchase    integer NOT NULL,
    ID_product    integer NOT NULL,
    quantity_in_purchase integer NOT NULL,
    CHECK (quantity_in_purchase > 0 AND quantity_in_purchase <= 100),
    price_in_purchase numeric(12,2) NOT NULL,
    status_item    varchar(30) NOT NULL,
    CHECK (status_item IN ('pending', 'shipped', 'delivered', 'cancelled','accepted', 'in progress')),

```

```

FOREIGN KEY ( ID_product ) REFERENCES Product ( ID_product )
    ON DELETE NO ACTION
    ON UPDATE CASCADE,
FOREIGN KEY ( ID_purchase ) REFERENCES Purchase ( ID_purchase )
    ON DELETE NO ACTION
    ON UPDATE CASCADE
);
--Статистика продаж за введенный период по типам искусства
CREATE OR REPLACE FUNCTION sales_by_art_type(start_date DATE, end_date DATE)
RETURNS TABLE (
    type_art VARCHAR(100),
    total_quantity BIGINT,
    total_revenue NUMERIC
) AS $$
BEGIN
    IF start_date IS NULL OR end_date IS NULL THEN
        RAISE EXCEPTION 'start_date и end_date не могут быть NULL';
    END IF;
    IF start_date > end_date THEN
        RAISE EXCEPTION 'Дата начала периода не может быть позже даты окончания';
    END IF;
    RETURN QUERY
    SELECT
        p.type_art,
        SUM(ip.quantity_in_purchase) AS total_quantity,
        SUM(ip.price_in_purchase * ip.quantity_in_purchase) AS total_revenue
    FROM Purchase pur
    JOIN Item_purchase ip ON pur.ID_purchase = ip.ID_purchase
    JOIN Product p ON ip.ID_product = p.ID_product
    WHERE pur.created_at BETWEEN start_date AND end_date
    AND ip.status_item IN ('accepted', 'delivered')
    GROUP BY p.type_art;
END;
$$ LANGUAGE plpgsql;
--Для создания индивидуального товара (на вход номер заказа, характеристики товара) (процедура
создает запись в таблицу товар и добавляет ID товара)
CREATE OR REPLACE PROCEDURE create_custom_product(
    p_order_id INTEGER,
    p_name VARCHAR(100),
    p_type_art VARCHAR(100),
    p_price NUMERIC(12,2)
)
AS $$
DECLARE
    v_seller_id INTEGER;
    v_buyer_id INTEGER;
    v_product_id INTEGER;
BEGIN
    IF p_name IS NULL OR LENGTH(TRIM(p_name)) = 0 THEN
        RAISE EXCEPTION 'PRODUCT_NAME_REQUIRED: Имя товара обязательно.';
    END IF;
    IF LENGTH(p_name) > 100 THEN
        RAISE EXCEPTION 'PRODUCT_NAME_TOO_LONG: Имя товара не может превышать 100
символов.';
    END IF;
    IF p_type_art IS NULL OR LENGTH(TRIM(p_type_art)) = 0 THEN
        RAISE EXCEPTION 'TYPE_ART_REQUIRED: Тип искусства обязателен.';
    END IF;
    IF LENGTH(p_type_art) > 100 THEN
        RAISE EXCEPTION 'TYPE_ART_TOO_LONG: Тип искусства не может превышать 100
символов.';
    END IF;
    IF p_price <= 0 THEN

```

```

        RAISE EXCEPTION 'INVALID_PRICE: Цена должна быть больше 0.';
    END IF;
    SELECT ID_seller, ID_buyer
    INTO v_seller_id, v_buyer_id
    FROM Production_purchase
    WHERE ID_production_purchase = p_order_id;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'ORDER_NOT_FOUND: Заказ на изготовление с ID % не найден.',
p_order_id;
    END IF;
    INSERT INTO Product (
        name, type_art, ID_seller, ID_indiv_buyer,
        quantity_for_sale, price, status
    ) VALUES (
        p_name, p_type_art, v_seller_id, v_buyer_id,
        1, p_price, 'reserved'
    )
    RETURNING ID_product INTO v_product_id;
    UPDATE Production_purchase
    SET ID_product = v_product_id
    WHERE ID_production_purchase = p_order_id;
    EXCEPTION
    WHEN UNIQUE_VIOLATION THEN
        RAISE EXCEPTION 'DUPLICATE_PRODUCT_DATA: Нарушена уникальность данных';
    WHEN OTHERS THEN
        RAISE EXCEPTION 'UNEXPECTED_ERROR: Ошибка при создании товара: %', SQLERRM;
END;
$$ LANGUAGE plpgsql;
--Получение списка материалов для товара
CREATE OR REPLACE FUNCTION get_materials_for_product(p_product_id INTEGER)
RETURNS TABLE (
    material_name VARCHAR(100),
    quantity NUMERIC(10,3),
    unit VARCHAR(20),
    total_cost NUMERIC
) AS $$
BEGIN
    IF p_product_id IS NULL THEN
        RAISE EXCEPTION 'PRODUCT_ID_REQUIRED: Необходимо указать ID товара.';
    END IF;
    PERFORM 1 FROM Product WHERE ID_product = p_product_id;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'PRODUCT_NOT_FOUND: Товар с ID % не существует.', p_product_id;
    END IF;
    RETURN QUERY
    SELECT
        m.material_name,
        cpm.quantity,
        cpm.unit,
        cpm.quantity * m.cost_per_unit AS total_cost
    FROM Connect_product_material cpm
    JOIN Catalog_for_material m ON cpm.ID_material = m.ID_material
    WHERE cpm.ID_product = p_product_id;
END;
$$ LANGUAGE plpgsql;
--При переходе заказа в состоянии 'принято' списать количество товара со склада
CREATE OR REPLACE FUNCTION deduct_stock_on_accept()
RETURNS TRIGGER AS $$
DECLARE
    is_individual BOOLEAN;
    current_qty INTEGER;
    new_qty INTEGER;
BEGIN

```

```

SELECT (ID_indiv_buyer IS NOT NULL), quantity_for_sale
INTO is_individual, current_qty
FROM Product
WHERE ID_product = NEW.ID_product;
IF is_individual THEN
    RETURN NEW;
END IF;
IF current_qty < NEW.quantity_in_purchase THEN
    RAISE EXCEPTION 'INSUFFICIENT_STOCK: Недостаточно товара "%". Доступно: % шт.,
запрошено: % шт.', NEW.ID_product, current_qty, NEW.quantity_in_purchase;
END IF;
new_qty := current_qty - NEW.quantity_in_purchase;
UPDATE Product
SET
    quantity_for_sale = new_qty,
    status = CASE WHEN new_qty = 0 THEN 'sold' ELSE 'available' END
WHERE ID_product = NEW.ID_product;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_deduct_on_accept
BEFORE UPDATE OF status_item ON Item_purchase
FOR EACH ROW
WHEN (NEW.status_item = 'accepted' AND OLD.status_item != 'accepted')
EXECUTE FUNCTION deduct_stock_on_accept();
--При добавлении товара в заказ проверять, что товара достаточно в наличии
CREATE OR REPLACE FUNCTION check_stock_on_insert()
RETURNS TRIGGER AS $$
DECLARE
    is_individual BOOLEAN;
    available_qty INTEGER;
BEGIN
    SELECT (ID_indiv_buyer IS NOT NULL), quantity_for_sale
    INTO is_individual, available_qty
    FROM Product
    WHERE ID_product = NEW.ID_product;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'PRODUCT_NOT_EXISTS: Товар с ID % не существует.', NEW.ID_product;
    END IF;
    IF is_individual THEN
        RETURN NEW;
    END IF;
    IF available_qty < NEW.quantity_in_purchase THEN
        RAISE EXCEPTION 'INSUFFICIENT_STOCK: Недостаточно товара "%". Доступно: %,
запрошено: %', (SELECT name FROM Product WHERE ID_product = NEW.ID_product), available_qty,
NEW.quantity_in_purchase;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_check_stock_insert
BEFORE INSERT ON Item_purchase
FOR EACH ROW
EXECUTE FUNCTION check_stock_on_insert();
--При добавлении состава товара выводить себестоимость общего изделия
CREATE TABLE IF NOT EXISTS Product_cost_log (
    log_id SERIAL PRIMARY KEY,
    ID_product INTEGER NOT NULL,
    total_cost NUMERIC(12,2) NOT NULL,
    updated_at TIMESTAMP DEFAULT NOW()
);
CREATE OR REPLACE FUNCTION log_product_cost()
RETURNS TRIGGER AS $$

```

```

DECLARE
    target_product_id INTEGER;
    total NUMERIC(12,2);
BEGIN
    IF TG_OP = 'DELETE' THEN
        target_product_id := OLD.ID_product;
    ELSE
        target_product_id := NEW.ID_product;
    END IF;
    IF target_product_id IS NULL THEN
        RETURN NULL;
    END IF;
    SELECT COALESCE(SUM(cpm.quantity * m.cost_per_unit), 0)
    INTO total
    FROM Connect_product_material cpm
    JOIN Catalog_for_material m ON cpm.ID_material = m.ID_material
    WHERE cpm.ID_product = target_product_id;
    INSERT INTO Product_cost_log (ID_product, total_cost)
    VALUES (target_product_id, total);
    IF TG_OP = 'DELETE' THEN
        RETURN OLD;
    ELSE
        RETURN NEW;
    END IF;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_log_cost_on_material_change
AFTER INSERT OR UPDATE OR DELETE ON Connect_product_material
FOR EACH ROW
EXECUTE FUNCTION log_product_cost();

```

Приложение Б. Перечень сокращений и обозначений

Модель класса Product

```
using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;

namespace RazorPagesMovie.Models;

public partial class Product
{
    public int IdProduct { get; set; }

    [Required(ErrorMessage = "Название товара обязательно")]
    [StringLength(100, ErrorMessage = "Название не должно превышать 100 символов")]
    [RegularExpression(@"^[A-Za-zА-Яа-я0-9\s\-,!~?]+\$",
        ErrorMessage = "Недопустимые символы в названии. Разрешены буквы, цифры, пробел и знаки препинания")]
    [Display(Name = "Название товара")]
    public string Name { get; set; } = null!;

    [Required(ErrorMessage = "Тип искусства обязателен")]
    [StringLength(100, ErrorMessage = "Тип искусства не должен превышать 100 символов")]
    [RegularExpression(@"^[A-Za-zА-Яа-я\s]+",
        ErrorMessage = "Тип искусства может содержать только буквы, пробел и дефис")]
    [Display(Name = "Тип искусства")]
    public string TypeArt { get; set; } = null!;

    [Display(Name = "ID продавца")]
    public int IdSeller { get; set; }

    [Range(0, 10000, ErrorMessage = "Количество для продажи должно быть от 0 до 10 000")]
    [Display(Name = "Количество товара")]
    public int QuantityForSale { get; set; }

    [Range(0.01, 999999999999.99, ErrorMessage = "Цена должна быть больше 0 и не превышать 999 999 999,99")]
    [Display(Name = "Цена")]
    public decimal Price { get; set; }

    [Required(ErrorMessage = "Статус товара обязателен")]
    [StringLength(30, ErrorMessage = "Статус не должен превышать 30 символов")]
    [RegularExpression(@"^(available|reserved|sold)$",
        ErrorMessage = "Допустимые значения статуса: available, reserved, sold (в нижнем регистре)")]
    [Display(Name = "Статус")]
    public string Status { get; set; } = null!;

    [Display(Name = "Индивид.товар покупателя")]
    public int? IdIndivBuyer { get; set; }

    public virtual ICollection<ConnectProductMaterial> ConnectProductMaterials { get; set; } = new List<ConnectProductMaterial>();

    public virtual Account? IdIndivBuyerNavigation { get; set; }

    public virtual Account? IdSellerNavigation { get; set; }

    public virtual ICollection<ItemPurchase> ItemPurchases { get; set; } = new List<ItemPurchase>();

    public virtual ICollection<ProductionPurchase> ProductionPurchases { get; set; } = new List<ProductionPurchase>();
}
```

Модуль класса добавления записи для таблицы Product

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.AspNetCore.Mvc.Rendering;
using Microsoft.EntityFrameworkCore;

namespace RazorPagesMovie.Pages.Product
{
    [Bind(
        "Product.Name",
        "Product.TypeArt",
        "Product.IdSeller",
        "Product.QuantityForSale",
        "Product.Price",
        "Product.Status",
        "Product.IdIndivBuyer"
    )]
    public class CreateModel : PageModel
    {
        private readonly RazorPagesMovie.Models.ArtMarketDbContext _context;

        public CreateModel(RazorPagesMovie.Models.ArtMarketDbContext context)
        {
            _context = context;
        }

        public IActionResult OnGet()
        {
            returnUrl = Request.Headers["Referer"].ToString() ?? "/Index";
            PopulateDropDowns();
            return Page();
        }

        [BindProperty]
        public Models.Product Product { get; set; } = default!;
        public string returnUrl { get; set; } = default!;

        public async Task<IActionResult> OnPostAsync()
        {
            //returnUrl = Request.Headers["Referer"].ToString() ?? "/Index";
            if (!ModelState.IsValid)
            {
                PopulateDropDowns();
                return Page();
            }

            if (Product.IdIndivBuyer == 0)
            {
                Product.IdIndivBuyer = null;
            }

            _context.Products.Add(Product);
            await _context.SaveChangesAsync();

            return RedirectToPage(returnUrl);
        }

        private void PopulateDropDowns()
        {
            // 1. Покупатели (IdIndivBuyer)
            var buyers = _context.Accounts
```

```

        .Include(a => a.IdRoleNavigation)
        .Where(a => a.IdRoleNavigation != null && a.IdRoleNavigation.RoleName ==
"buyer")
        .ToList();

var buyersSelectList = new SelectList(buyers, "IdAccount", "AccountName");
var items = buyersSelectList.ToList();

items.Insert(0, new SelectListItem
{
    Value = "",
    Text = "— Выберите покупателя индивидуального заказа (Необязательно) —",
    Selected = true
});

ViewData["IdIndivBuyer"] = items;

// 2. Продавцы (IdSeller)
var sellers = _context.Accounts
    .Include(a => a.IdRoleNavigation)
    .Where(a => a.IdRoleNavigation != null && a.IdRoleNavigation.RoleName ==
"seller")
    .ToList();

ViewData["IdSeller"] = new SelectList(sellers, "IdAccount", "AccountName");
    }
}
}

```

Модель класса авторизации

```

using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Authentication.Cookies;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.EntityFrameworkCore;
//using Microsoft.VisualStudio.Web.CodeGenerators.Mvc.Templates.BlazorIdentity.Pages.Manage;
using RazorPagesMovie.Models;
//using System.Data;
using System.Security.Claims;

```

```

namespace RazorPagesMovie.Pages
{
    public class AuthorizationModel : PageModel
    {
        private readonly ArtMarketDbContext _context;

        public AuthorizationModel(ArtMarketDbContext context)
        {
            _context = context;
        }

        [BindProperty]
        public string LoginInput { get; set; } = string.Empty;

        [BindProperty]
        public string Password { get; set; } = string.Empty;

        public string ErrorMessage { get; set; } = string.Empty;

        public void OnGet()
        {
        }
    }
}

```



```

public async Task<IActionResult> OnPostAsync()
{
    if (string.IsNullOrEmpty(LoginInput) || string.IsNullOrEmpty>Password))
    {
        ErrorMessage = "Введите имя/email и пароль.";
        return Page();
    }

    var user = await _context.Accounts
        .Include(a => a.IdRoleNavigation)
        .FirstOrDefaultAsync(a =>
            (a.AccountName == LoginInput || a.Email == LoginInput) &&
            a.Password == Password);

    if (user == null) { Console.WriteLine($"ПОЛЬЗОВАТЕЛЬ НЕ НАЙДЕН!
LoginInput='{LoginInput}', Password='{Password}"); }
    else {
        string roleName = user.IdRoleNavigation?.RoleName ?? "null";
        Console.WriteLine($"НАЙДЕН ПОЛЬЗОВАТЕЛЬ! Имя='{user.AccountName}',
Email='{user.Email}', Роль='{roleName}");
    }

    if (user != null && user.IdRoleNavigation != null)
    {
        string accountName = user.AccountName ?? "null";
        string roleName = user.IdRoleNavigation?.RoleName ?? "null";
        string email = user.Email;
        int IdAccount = user.IdAccount;

        var claims = new List<Claim>
        {
            new(ClaimTypes.Name, accountName),
            new(ClaimTypes.Email, email),
            new(ClaimTypes.Role, roleName),
            new("idaccount", IdAccount.ToString())
        };
        var identity = new ClaimsIdentity(claims, CookieAuthenticationDefaults.AuthenticationScheme);
        var principal = new ClaimsPrincipal(identity);

        var authProperties = new AuthenticationProperties
        {
            IsPersistent = true,
            ExpiresUtc = DateTimeOffset.UtcNow.AddMinutes(15)
        };
        await HttpContext.SignInAsync(CookieAuthenticationDefaults.AuthenticationScheme, principal,
authProperties);

        Console.WriteLine($"Перенаправление по роли: {user.IdRoleNavigation.RoleName}");

        return user.IdRoleNavigation.RoleName switch
        {
            "admin" => RedirectToPage("/Admin"),
            "buyer" => RedirectToPage("/Index"),
            "seller" => RedirectToPage("/Seller/Index"),
            _ => RedirectToPage("/Authorization")
        };
    }

    ErrorMessage = "Неверное имя/email или пароль.";
    return Page();
}

```

```

    }
  }
}

Страница чата между продавцом и покупателем
@page
@attribute [Microsoft.AspNetCore.Authorization.Authorize(Roles = "admin,seller,buyer")]
@model RazorPagesMovie.Pages.Chat.RoomModel

<h1>Чат продавца с покупателем</h1>

<div class="container-sm mt-4" style="max-width: 900px;">
  <div class="card mb-4">
    <div class="card-header bg-info text-white">
      <h2 class="h5 mb-0">Переписка Покупателя с Продавцом</h2>
    </div>

    <!-- Два блока: Продавец и Покупатель -->
    <div class="d-flex">
      <div class="flex-fill text-center p-2 border">
        Продавец: <strong>"@Model.SellerUser.AccountName"</strong>
        @if (@Model.CurrentUser.IdRoleNavigation.RoleName == "seller") { <span>(Это Вы)</span>
      }

      </div>
      <div class="flex-fill text-center p-2 border">
        Покупатель: <strong>"@Model.BuyerUser.AccountName"</strong>
        @if (@Model.CurrentUser.IdRoleNavigation.RoleName == "buyer") { <span>(Это Вы)</span>
      }

    </div>
  </div>
  </div>
  </div>
  <div class="card mb-4">
    <div class="card-body" style="max-height: 400px; overflow-y: auto;">
      @if (Model.ChatMessages.Count==0)
      {
        <p class="text-muted">Переписка пока пуста. Напишите первое сообщение!</p>
      }
      else
      {
        <div class="messages">
          @foreach (var message in Model.ChatMessages)
          {
            <div class="message mb-3 p-3 rounded @((message.DirectionFromSeller) ? "bg-success" : "bg-
primary text-white ms-auto)" style="max-width: 80%;">
              <div class="message-header small fw-bold mb-1">
                @if (message.DirectionFromSeller)
                { <span>(Продавец: @Model.SellerUser.AccountName)</span>
                } else {
                  <span>(Покупатель: @Model.BuyerUser.AccountName)</span>
                }
              </div>
              <div class="message-content">
                @message.TextAccounts
              </div>
            </div>
          }
        </div>
      }
    </div>
  </div>
</div>

```

@* Используем ms-auto для выравнивания вправо (Bootstrap) и тернарный оператор для цвета *@

```

        <div class="card @(Model.CurrentUser.IdRoleNavigation.RoleName == "buyer" ? "ms-auto" : "")"
style="width: 80%;">
            <div class="card-header" style="background-color:
@(Model.CurrentUser.IdRoleNavigation.RoleName == "buyer" ? "#0d6efd" : "#22dd77"); color: white;">
                <h2 class="h5 mb-0">Новое сообщение</h2>
            </div>
            <div class="card-body">
                <form method="post">
                    <div class="mb-3">
                        <textarea asp-for="NewMessage" class="form-control" rows="3" placeholder="Введите
ваше сообщение..."></textarea>
                        <span asp-validation-for="NewMessage" class="text-danger"></span>
                    </div>
                    <button type="submit" class="btn btn-success">
                        <i class="bi bi-send"></i> Отправить сообщение
                    </button>
                </form>
            </div>
        </div>

```

```

</div>

```

```

<style>
.messages {
    display: flex;
    flex-direction: column;
}

.message {
    margin-bottom: 1rem;
    word-wrap: break-word;
}

.message-header {
    font-size: 0.85em;
}

.message-content {
    line-height: 1.4;
}
</style>

```

```

@section Scripts {
    @{
        await Html.RenderPartialAsync("_ValidationScriptsPartial");
    }
    <script>
        // Автоматическая прокрутка к последнему сообщению
        document.addEventListener('DOMContentLoaded', function() {
            const messageContainer = document.querySelector('.card-body[style*="max-height"]');
            if (messageContainer) {
                messageContainer.scrollTop = messageContainer.scrollHeight;
            }
        });

        // Автоматическая прокрутка после отправки сообщения
        document.querySelector('form').addEventListener('submit', function() {
            setTimeout(function() {
                const container = document.querySelector('.card-body[style*="max-height"]');
                if (container) {
                    container.scrollTop = container.scrollHeight;
                }
            }, 100);
        });
    </script>

```

```

    }
    }, 500);
  });
});
</script>
}

```

Модуль класса чата между покупателем и продавцом

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.EntityFrameworkCore;
using RazorPagesMovie.Models;

namespace RazorPagesMovie.Pages.Chat
{
    public class RoomModel : PageModel
    {
        private readonly ArtMarketDbContext _context;
        public RoomModel(ArtMarketDbContext context)
        {
            _context = context;
        }
        public List<Models.ProductionPurchase> ChatMessages { get; set; } = new();
        public Models.Account CurrentUser { get; set; }
        public Models.Account SellerUser { get; private set; }
        public Models.Account BuyerUser { get; private set; }
        public int? CurrentUserId { get; private set; }

        [BindProperty]
        public string NewMessage { get; set; }

        public async Task OnGetAsync(int idSeller, int idBuyer)
        {
            Console.WriteLine($"Начало чата между idSeller: {idSeller}, idBuyer: {idBuyer}");

            if (int.TryParse(User.FindFirst("idaccount")?.Value, out int id))
            {
                CurrentUserId = id;
            }
            SellerUser = await _context.Accounts.FindAsync(idSeller);
            BuyerUser = await _context.Accounts.FindAsync(idBuyer);
            CurrentUser = await _context.Accounts
                .Include(a => a.IdRoleNavigation)
                .FirstOrDefaultAsync(a => a.IdAccount == CurrentUserId);

            ChatMessages = await _context.ProductionPurchases
                .Include(p => p.IdBuyerNavigation)
                .Include(p => p.IdSellerNavigation)
                .Where(p => p.IdSeller == idSeller)
                .Where(p => p.IdBuyer == idBuyer)
                .ToListAsync();
        }

        public async Task<IActionResult> OnPostAsync(int idSeller, int idBuyer)
        {
            if (string.IsNullOrEmpty(NewMessage))
            {
                return RedirectToPage(new { idSeller, idBuyer });
            }

            if (int.TryParse(User.FindFirst("idaccount")?.Value, out int id))

```

```

    {
        CurrentUserId = id;
    }

    var message = new Models.ProductionPurchase
    {
        IdSeller = idSeller,
        IdBuyer = idBuyer,
        TextAccounts = NewMessage,
        DirectionFromSeller = (CurrentUserId == idSeller)
    };

    _context.ProductionPurchases.Add(message);
    await _context.SaveChangesAsync();

    return RedirectToPage(new { idSeller, idBuyer });
}
}

```